

LAPORAN TUGAS BESAR 2
IF3270 PEMBELAJARAN MESIN
Convolutional Neural Network dan Recurrent Neural Network



Disusun oleh:

Kelompok 9 ciamsoV2

Anella Utari Gunadi	13523078
Nayla Zahira	13523079
Diyah Susan Nugrahani	13523080

SEKOLAH TINGGI ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

BAB 1	4
DESKRIPSI PERSOALAN	4
BAB 2	5
PEMBAHASAN	5
2.1 Penjelasan Implementasi	5
2.1.1 Struktur Kode Program	5
2.1.2 Kelas CNN	5
2.1.10 Kelas EmbeddingScratch	9
2.1.11 Kelas DenseScratch	9
2.1.12 Kelas SimpleRNNCellScratch	9
2.1.13 Kelas SimpleRNNLayerScratch	10
2.1.14 Kelas RNNCaptionDecoderScratch	10
2.1.15 Kelas EmbeddingLayer	11
2.1.16 Kelas DenseLayer	12
2.1.17 Kelas LSTMLayer	12
2.1.18 Kelas LSTMFromScratch	13
2.2 Forward Propagation CNN	13
2.2.1 Input Layer	13
2.2.2 Convolutional Layer (Conv2D)	14
2.2.4 Pooling Layer	15
2.2.5 Flattening Layer	17
2.2.6 Dense Layer	17
2.2.7 Locally Connected Layer	17
2.2.8 Kelas CNN	18
2.3 Forward Propagation RNN	19
2.3.1. EmbeddingScratch	20
2.3.2. DenseScratch	20
2.3.3. SimpleRNNCellScratch	20
2.3.4. SimpleRNNLayerScratch	21
2.3.5. RNNCaptionDecoderScratch	22
2.4 Forward Propagation LSTM	24
2.4.1. Kelas Embedding Layer	24
2.4.2. Kelas Dense Layer	25
2.4.3. Kelas LSTM Layer	25
2.4.4. Kelas LSTMFromScratch	27
BAB 3	29
HASIL PENGUJIAN	29
3.1 Hasil Pengujian CNN	29

3.1.1 Perbandingan shared vs non-shared parameter	29
3.1.2 Pengaruh jumlah layer konvolusi	32
3.1.3 Pengaruh banyak filter per layer konvolusi	38
3.1.4 Pengaruh ukuran filter per layer konvolusi	38
3.1.5 Pengaruh jenis pooling layer	38
3.1.6 Perbandingan Keras vs from scratch	38
3.2 Hasil Pengujian Simple RNN dan LSTM	39
3.2.1. Perbandingan jumlah layer: RNN	39
3.2.2. Perbandingan jumlah layer: LSTM	41
3.2.3. Perbandingan ukuran hidden state: RNN	42
3.2.4. Perbandingan ukuran hidden state: LSTM	44
3.2.5. Perbandingan RNN vs LSTM	45
3.2.6. Perbandingan Keras vs From Scratch	48
3.2.7. Pengaruh panjang maksimum caption terhadap BLEU-4	49
BAB 4	51
KESIMPULAN DAN SARAN	51
4.1. Kesimpulan	51
4.2. Saran	52
PEMBAGIAN TUGAS	53
REFERENSI	54
LAMPIRAN	55

BAB 1

DESKRIPSI PERSOALAN

Pada Tugas Besar 2 IF3270 Pembelajaran Mesin ini, persoalan utama yang harus diselesaikan adalah merancang dan mengimplementasikan arsitektur *Convolutional Neural Network* (CNN) dan *Recurrent Neural Network* (RNN). Mahasiswa dihadapkan pada tantangan teknis untuk membangun modul *forward propagation* untuk arsitektur CNN, Simple RNN, dan *Long Short-Term Memory* (LSTM) sepenuhnya *from scratch*. dengan menggunakan bahasa python dan memanfaatkan *library* Numpy.

Implementasi CNN tersebut akan difokuskan pada penyelesaian masalah klasifikasi gambar (*image classification*) menggunakan dataset Intel Image Classification. Persoalan ini juga meluas pada eksplorasi *pipeline image captioning* pada dataset Flickr8k. *Pipeline* ini dibangun melalui arsitektur *encoder-decoder* yang mengintegrasikan CNN sebagai *encoder* dan RNN/LSTM sebagai *decoder*.

Pada tahap akhir, seluruh implementasi ini diuji melalui evaluasi yang komprehensif. Peserta diharuskan memecahkan persoalan komparatif dengan membandingkan metrik performa, seperti *macro F1-score* untuk klasifikasi dan *BLEU-4* serta *METEOR score* untuk *image captioning*. Lalu dilakukan perbandingan hasil implementasi *from scratch* dengan *framework* Keras, serta menganalisis perbedaan kemampuan memori jangka panjang antara RNN dan LSTM secara kuantitatif maupun kualitatif.

BAB 2

PEMBAHASAN

2.1 Penjelasan Implementasi

2.1.1 Struktur Kode Program

TUBES2_IF3270_KELOMPOK9	
├── data/	# Folder untuk dataset
├── doc/	
│ ├── Tubes2_IF3270_Kelompok.pdf	# Laporan tugas besar
├── src/	# Folder source code utama
│ ├── cnn/	# Implementasi Forward Propagation CNN
│ ├── lstm/	# Implementasi Forward Propagation LSTM
│ ├── rnn/	# Implementasi Forward Propagation RNN
│ ├── weights/	# Saved weights dari model
└── README.md	# Dokumentasi proyek

2.1.2 Kelas CNN

Kelas CNN digunakan sebagai kelas *wrapping* yang mewadahi layer-layer yang akan digunakan pada model CNN. Atributnya berupa *list of layers*.

Atribut	
self.layers	List of layers
Metode	
__init__(self, layers)	<i>method</i> untuk inisialisasi kelas
forward(self, x)	<i>method</i> untuk melakukan <i>forward propagation</i> pada setiap layer yang ada
save_weights	<i>method</i> untuk menyimpan bobot
load_weights	<i>method</i> untuk <i>load</i> bobot

2.1.3 Kelas Conv2D

Kelas Conv2D merupakan kelas untuk layer konvolusi. Kelas ini berfungsi untuk melakukan ekstraksi fitur menghasilkan *feature map*.

Atribut	
self.kernel	Matriks bobot atau kernel yang digunakan untuk pembelajaran
self.bias	Bias pembelajaran

self.stride	Ukuran pergeseran
self.padding	Ukuran <i>padding</i> dari input gambar
self.kH	Ukuran <i>height</i> kernel
self.kW	Ukuran <i>width</i> kernel
self.C_in	<i>Channel input</i>
self.C_out	<i>Channel output</i>
Metode	
<code>__init__(self, layers)</code>	<i>method</i> untuk inisialisasi kelas
<code>forward(self, input_tensor)</code>	<i>method</i> untuk melakukan <i>forward propagation</i> layer konvolusi

2.1.4 Kelas LocallyConnected2D

Kelas LocallyConnected2D merupakan kelas untuk model *non-shared parameter*. Cara kerja dari kelas ini mirip dengan Conv2D, hanya saja perhitungan pada setiap pergeseran dilakukan dengan kernel yang berbeda-beda.

Atribut	
self.kernel	Matriks bobot atau kernel yang digunakan untuk pembelajaran
self.bias	Bias pembelajaran
self.stride	Ukuran pergeseran
self.kH	Ukuran <i>height</i> kernel
self.kW	Ukuran <i>width</i> kernel
Metode	
<code>__init__(self, layers)</code>	<i>method</i> untuk inisialisasi kelas
<code>forward(self, input_tensor)</code>	<i>method</i> untuk melakukan <i>forward propagation</i> layer <i>locally connected</i>

2.1.5 Kelas MaxAvgPooling2D

Kelas MaxAvgPooling2D merupakan kelas untuk layer *pooling* yang dilakukan dengan memperhatikan ukuran *pool size* dengan dua tipe pilihan *avg* dan *max pooling*.

Atribut	
self.pool_size	Ukuran matriks untuk <i>pooling</i>
self.stride	Ukuran pergeseran
self.tipe	Tipe <i>pooling</i> yang dipilih, bisa pilih <i>max</i> atau <i>avg</i>
Metode	
<code>__init__(self, layers)</code>	<i>method</i> untuk inisialisasi kelas
<code>forward(self, input_tensor, tipe=None)</code>	<i>method</i> untuk melakukan <i>forward propagation</i> layer <i>pooling</i>

2.1.6 Kelas GlobalMaxAvgPooling2D

Kelas GlobalMaxAvgPooling2D merupakan variasi *pooling* yang dilakukan secara global tanpa memperhitungkan ukuran *pool size* maupun *stride* karena *pooling* dilakukan untuk langsung untuk seluruh matriks.

Atribut	
self.tipe	Tipe <i>pooling</i> yang dipilih, bisa pilih <i>max</i> atau <i>avg</i>
Metode	
<code>__init__(self, layers)</code>	<i>method</i> untuk inisialisasi kelas
<code>forward(self, input_tensor, tipe=None)</code>	<i>method</i> untuk melakukan <i>forward propagation</i> layer <i>global pooling</i>

2.1.7 Kelas Flatten

Kelas Flatten merupakan kelas yang digunakan untuk mengubah matriks multidimensi menjadi vektor satu dimensi agar nantinya bisa diteruskan ke *dense layer*.

Atribut	
-	-
Metode	
forward(self, input_tensor)	<i>method</i> untuk melakukan <i>forward propagation</i> layer <i>flatten</i>

2.1.8 Kelas Dense

Kelas Dense merupakan kelas yang digunakan untuk melakukan perhitungan menggunakan input dari layer-layer sebelumnya dengan metode *fully connected*.

Atribut	
self.kernel	Matriks bobot atau kernel yang digunakan untuk pembelajaran
self.bias	Bias pembelajaran
Metode	
<code>__init__(self, weights, bias)</code>	<i>method</i> untuk inisialisasi kelas
forward(self, input_tensor)	<i>method</i> untuk melakukan <i>forward propagation</i> layer <i>dense</i>

2.1.9 Kelas FungsiAktivasi

Kelas FungsiAktivasi merupakan kelas yang digunakan untuk memilih fungsi aktivasi apa yang ingin digunakan pada model CNN.

Atribut	
self.tipe	Tipe untuk memilih fungsi aktivasi yang digunakan. Bisa memilih tipe relu, sigmoid, tanh, atau softmax.
Metode	
<code>__init__(self,</code>	<i>method</i> untuk inisialisasi kelas

tipe='relu')	
forward(self, input_tensor, tipe=None)	<i>method</i> untuk melakukan <i>forward propagation layer activation function</i>

2.1.10 Kelas EmbeddingScratch

Kelas EmbeddingScratch digunakan untuk mengubah token caption berbentuk indeks integer menjadi representasi vektor dense.

Atribut	
self.weights: dictionary yang menyimpan matriks embedding	
Method	
__init__(self, weights: np.ndarray)	Method untuk inisialisasi embedding layer
forward(self, token_ids: np.ndarray) -> np.ndarray	Method untuk mengambil embedding berdasarkan token input

2.1.11 Kelas DenseScratch

Kelas DenseScratch digunakan untuk mengubah token caption berbentuk indeks integer menjadi representasi vektor dense.

Atribut	
self.W: variabel untuk menyimpan bobot self.b: variabel untuk menyimpan bias self.activation: variabel untuk menyimpan fungsi aktivasi	
Method	
__init__(self, W: np.ndarray, b: np.ndarray, activation: str None = None)	Method untuk inisialisasi dense layer
forward(self, x: np.ndarray)	Method untuk melakukan forward propagation, menghitung kombinasi linear input bobot dan bias lalu menerapkan fungsi aktivasinya.

2.1.12 Kelas SimpleRNNCellScratch

Kelas SimpleRNNCellScratch digunakan untuk

Atribut

self.Wx: variabel untuk menyimpan bobot input self.Wh: variabel untuk menyimpan bobot hidden state antar timestep self.b: variabel untuk menyimpan bias self.activation: variabel untuk menyimpan fungsi aktivasi self.units: variabel untuk menyimpan jumlah unit <i>hidden state</i>	
Method	
__init__(self, kernel: np.ndarray, recurrent_kernel: np.ndarray, bias: np.ndarray, activation: str = "tanh"):	Method untuk inisialisasi cell Simple RNN dengan menyimpan bobot input, bobot recurrent, bias, fungsi aktivasi, dan jumlah unit hidden state.
step(self, x_t: np.ndarray, h_prev: np.ndarray) -> np.ndarray	Method untuk menghitung hidden state baru pada satu timestep berdasarkan input saat ini dan hidden state sebelumnya.

2.1.13 Kelas SimpleRNNLayerScratch

Kelas SimpleRNNLayerScratch digunakan untuk

Atribut	
self.cell: variabel untuk menyimpan objek SimpleRNNCellScratch yang digunakan pada setiap timestep self.return_sequences: variabel untuk menentukan layer mengembalikan seluruh output sequence atau hanya hidden state terakhir self.units: variabel untuk menyimpan jumlah unit hidden state pada layer RNN	
Method	
__init__(self, kernel: np.ndarray, recurrent_kernel: np.ndarray, bias: np.ndarray, activation: str = "tanh", return_sequences: bool = True)	Method untuk inisialisasi layer Simple RNN dengan membuat objek SimpleRNNCellScratch, menyimpan konfigurasi return_sequences, dan menyimpan jumlah unit hidden state.
forward(self, x: np.ndarray, h0: np.ndarray None = None)	Method untuk melakukan forward propagation pada seluruh sequence input. Hidden state awal diinisialisasi dengan nol jika tidak diberikan, lalu setiap timestep diproses menggunakan method step().

2.1.14 Kelas RNNCaptionDecoderScratch

Kelas RNNCaptionDecoderScratch digunakan untuk

Atribut

self.feature_projection: variabel untuk menyimpan layer dense proyeksi fitur gambar self.embedding: variabel untuk menyimpan embedding layer token caption self.rnn_layers: variabel untuk menyimpan kumpulan layer Simple RNN self.output_dense: variabel untuk menyimpan dense output layer untuk prediksi vocabulary self.idx_to_word: variabel untuk menyimpan mapping indeks ke kata self.word_to_idx: variabel untuk menyimpan mapping kata ke indeks	
Method	
__init__(self, kernel: np.ndarray, recurrent_kernel: np.ndarray, bias: np.ndarray, activation: str = "tanh", return_sequences: bool = True)	Method untuk inisialisasi decoder RNN dengan menyimpan layer proyeksi fitur gambar, embedding caption, kumpulan layer RNN, dense output layer, serta mapping indeks-kata dan kata-indeks.
_make_preinject_sequence(self, image_features: np.ndarray, input_token_ids: np.ndarray)	Method untuk membentuk input sequence dengan metode pre-inject. Fitur gambar diproyeksikan terlebih dahulu menjadi embedding, kemudian digabung di depan embedding token caption sebagai timestep awal.
forward_teacher_forcing(self, image_features: np.ndarray, input_token_ids: np.ndarray)	Method untuk menjalankan proses prediksi saat training/evaluasi dengan caption input yang sudah diketahui. Data masuk ke RNN, lalu hasilnya diubah menjadi probabilitas kata pada vocabulary.
predict_next_distribution(self, image_features: np.ndarray, input_token_ids: np.ndarray)	Method untuk mengambil probabilitas kata berikutnya berdasarkan gambar dan token caption yang sudah masuk sebelumnya.
generate_greedy(self, image_feature: np.ndarray, start_id: int, end_id: int, max_len: int = 30, pad_id: int None = None)	Method untuk membuat caption secara bertahap dengan memilih kata yang memiliki probabilitas paling besar pada setiap langkah.
generate_caption(self, image_feature: np.ndarray, start_id: int, end_id: int, max_len: int = 30, pad_id: int None = None)	Method untuk menghasilkan caption akhir dalam bentuk kalimat, bukan lagi indeks angka.

2.1.15 Kelas EmbeddingLayer

Kelas EmbeddingLayer digunakan untuk mengubah token caption berbentuk indeks integer menjadi representasi vektor dense.

Atribut	
self.weights: dictionary yang menyimpan matriks embedding	
Method	
__init__(self, weights)	Method untuk inisialisasi embedding layer
forward(self, x)	Method untuk mengambil embedding berdasarkan token input

2.1.16 Kelas DenseLayer

Kelas DenseLayer digunakan untuk melakukan transformasi linear dan menghasilkan output vocabulary pada decoder LSTM.

Atribut	
self.weights : dictionary yang menyimpan kernel dan bias self.activation : variabel untuk menyimpan jenis fungsi aktivasi yang digunakan	
Metode	
__init__(self, weights, activation=None)	Method untuk inisialisasi dense layer
_softmax(self, x)	Method untuk menghitung softmax
forward(self, x)	Method untuk melakukan forward propagation dense layer

2.1.17 Kelas LSTMLayer

Kelas LSTMLayer digunakan untuk memproses data sekuensial menggunakan mekanisme hidden state dan cell state pada LSTM.

Atribut	
self.units : jumlah hidden unit self.return_sequences : boolean untuk menentukan apakah output seluruh sequence atau hanya hidden terakhir self.weights : dictionary untuk menyimpan bobot kernel, recurrent kernel, dan bias	
Metode	
__init__(self, units, return_sequences=True, weights=None)	Method untuk inisialisasi layer LSTM
_sigmoid(self, x)	Method untuk menghitung fungsi sigmoid

<code>_tanh(self, x)</code>	Method untuk menghitung fungsi tanh
<code>forward(self, x)</code>	Method untuk melakukan forward propagation pada sequence input

2.1.18 Kelas LSTMFromScratch

Kelas LSTMFromScratch merupakan *wrapper class* yang mengintegrasikan embedding layer, feature projection layer, layer LSTM, dan output layer menjadi satu pipeline.

Atribut	
<code>self.embedding_layer</code> : layer embedding token caption <code>self.feature_projection_layer</code> : layer untuk memproyeksikan feature CNN <code>self.lstm_layers</code> : list layer yang digunakan <code>self.output_layer</code> : dense output layer vocabulary <code>self.word_to_idx</code> : mapping kata ke indeks <code>self.idx_to_word</code> : mapping indeks ke kata <code>self.max_len</code> : panjang maksimum caption <code>self.start_id</code> : token awal caption <code>self.end_id</code> : token akhir caption	
Metode	
<code>__init__(self, weights, activation=None)</code>	Method untuk inisialisasi
<code>forward(self, cnn_feature, input_tokens)</code>	Method untuk melakukan forward propagation captioning
<code>greedy_decode(self, cnn_feature)</code>	Method untuk menghasilkan caption menggunakan greedy decoding

2.2 Forward Propagation CNN

Forward Propagation pada CNN merupakan proses mengalirkan data gambar dari lapisan input melewati beberapa *hidden layer* hingga mencapai lapisan *output* untuk menghasilkan sebuah prediksi. Berbeda dengan *Neural Network* biasa yang langsung meratakan data, CNN memproses data dalam bentuk matriks untuk mempertahankan struktur spasial gambar.

2.2.1 Input Layer

Tahap paling awal adalah memproses input gambar menjadi bentuk representasi matriks angka. Untuk gambar berwarna (RGB), matriks ini memiliki dimensi tinggi, lebar, dan

tiga *channel* warna. Nilai piksel kemudian dinormalisasikan menjadi rentang 0 hingga 1 untuk memudahkan komputasi.

```
from PIL import Image

def siapkan_gambar(image_path):
    img = Image.open(image_path).convert('RGB').resize((150, 150))
    img_array = np.array(img)
    img_array = img_array / 255.0

    return img_array.astype(np.float32)
```

2.2.2 Convolutional Layer (Conv2D)

Lapisan ini merupakan inti dari CNN yang berfungsi untuk mengekstraksi fitur seperti garis hingga tekstur dari gambar. Operasi utama pada layer ini adalah konvolusi dengan cara menggeser kernel ke seluruh bagian gambar. Pada setiap pergeseran, dilakukan operasi *dot product* antara bobot kernel dan piksel gambar lalu ditambahkan dengan nilai bias. Hasil dari operasi ini akan menghasilkan *feature map*.

```
class Conv2D:
    def __init__(self, kernel_weights, bias_weights, stride=1,
                 padding=0):
        self.kernel = kernel_weights
        self.bias = bias_weights
        self.stride = stride
        self.padding = padding

        self.kH, self.kW, self.C_in, self.C_out = self.kernel.shape

    def forward(self, input_tensor):
        if self.padding > 0:
            p = self.padding
            input_tensor = np.pad(input_tensor, ((p, p), (p, p), (0, 0)), mode='constant', constant_values=0)

        H, W, C_in_input = input_tensor.shape

        assert C_in_input == self.C_in, "Jumlah channel input tidak sesuai dengan kernel"

        # hitung output dimensions
        out_H = int((H - self.kH) / self.stride) + 1
        out_W = int((W - self.kW) / self.stride) + 1
        output_tensor = np.zeros((out_H, out_W, self.C_out))

        output_tensor = np.zeros((out_H, out_W, self.C_out))
```

```

for h in range(out_H):
    for w in range(out_W):
        # batas window/patch pada gambar
        h_start = h * self.stride
        h_end = h_start + self.kH
        w_start = w * self.stride
        w_end = w_start + self.kW

        patch = input_tensor[h_start:h_end, w_start:w_end, :]

        for k in range(self.C_out):
            kernel_k = self.kernel[:, :, :, k]
            output_tensor[h, w, k] = np.sum(patch * kernel_k)
            + self.bias[k]

return output_tensor

```

2.2.3 Activation Function

Setelah konvolusi, *feature map* akan masuk ke fungsi aktivasi untuk memberikan sifat non-linearitas ke dalam model. Tanpa non-linearitas, CNN tidak akan bisa memecahkan pola data yang kompleks. Fungsi yang diimplementasikan untuk tugas besar ini antara lain relu, sigmoid, tanh, dan softmax. Fungsi yang paling umum digunakan dan terbukti memberikan hasil yang bagus adalah ReLU.

```

class FungsiAktivasi:
    def __init__(self, tipe='relu'):
        self.tipe = tipe

    def forward(self, input_tensor, tipe=None):
        if tipe is None:
            tipe = self.tipe

        if tipe == 'relu':
            return np.maximum(0, input_tensor)
        elif tipe == 'sigmoid':
            return 1 / (1 + np.exp(-input_tensor))
        elif tipe == 'tanh':
            return np.tanh(input_tensor)
        elif tipe == 'softmax':
            exp_values = np.exp(input_tensor - np.max(input_tensor))
            return exp_values / np.sum(exp_values)

```

2.2.4 Pooling Layer

Lapisan *pooling* berfungsi untuk melakukan *downsampling* atau pengurangan dimensi spasial namun tetap mempertahankan informasi penting di dalamnya. Hal ini dilakukan

untuk mengurangi beban komputasi dan mencegah model mengalami *overfitting*. Pada tugas besar ini, terdapat dua jenis *pooling* yang diimplementasikan yaitu *average pooling* dan *max pooling*.

```
class MaxAvgPooling2D:
    def __init__(self, pool_size=2, stride=2, tipe='max'):
        self.pool_size = pool_size
        self.stride = stride
        self.tipe = tipe

    def forward(self, input_tensor, tipe=None):
        if tipe is None:
            tipe = self.tipe

        H, W, C = input_tensor.shape
        pool_h = self.pool_size[0] if isinstance(self.pool_size,
        tuple) else self.pool_size
        pool_w = self.pool_size[1] if isinstance(self.pool_size,
        tuple) else self.pool_size

        out_H = int((H - pool_h) / self.stride) + 1
        out_W = int((W - pool_w) / self.stride) + 1

        output_tensor = np.zeros((out_H, out_W, C))
        for h in range(out_H):
            for w in range(out_W):
                h_start = h * self.stride
                h_end = h_start + pool_h
                w_start = w * self.stride
                w_end = w_start + pool_w

                patch = input_tensor[h_start:h_end, w_start:w_end, :]
                if tipe == 'max':
                    output_tensor[h, w, :] = np.max(patch, axis=(0,
                    1))
                elif tipe == 'avg':
                    output_tensor[h, w, :] = np.mean(patch, axis=(0,
                    1))

        return output_tensor

class GlobalMaxAvgPooling2D:
    def __init__(self, tipe='max'):
        self.tipe = tipe

    def forward(self, input_tensor, tipe=None):
        if tipe is None:
            tipe = self.tipe

        if tipe == 'max':
            return np.max(input_tensor, axis=(0, 1))
        elif tipe == 'avg':
```



```
return np.mean(input_tensor, axis=(0, 1))
```

2.2.5 Flattening Layer

Lapisan terakhir pada CNN biasanya merupakan *dense layer*. Matriks multidimensi dari lapisan *pooling* terakhir harus diubah bentuknya menjadi sebuah vektor satu dimensi agar dapat diproses ke *dense layer*. Proses ini disebut sebagai *flattening*.

```
class Flatten:
    def forward(self, input_tensor):
        return input_tensor.flatten()
```

2.2.6 Dense Layer

Vektor hasil *flattening* akan digunakan sebagai input pada layer ini. Setiap neuron di lapisan ini terhubung dengan setiap neuron di lapisan sebelumnya, atau sering juga disebut sebagai *fully connected layer*. Lapisan ini bertugas menyusun berbagai pola tingkat tinggi yang sudah diekstraksi oleh lapisan sebelumnya untuk menentukan keputusan klasifikasi.

```
class Dense:
    def __init__(self, weights, bias):
        self.kernel = weights
        self.bias = bias

    def forward(self, input_tensor):
        expected_input_dim = self.kernel.shape[0]
        actual_input_dim = input_tensor.shape[0]

        assert actual_input_dim == expected_input_dim, f"ERROR DENSE:
        Harapannya input ukuran {expected_input_dim}, tapi dapat
        {actual_input_dim}!"

        output = np.dot(input_tensor, self.kernel) + self.bias

        return output
```

2.2.7 Locally Connected Layer

Locally Connected Layer adalah variasi arsitektur di mana prinsip *parameter sharing* dihilangkan. Pada lapisan ini, operasi *dot product* dilakukan dengan cara menggunakan matriks kernel yang berbeda untuk setiap posisi spasial di *feature map*. Hal ini menyebabkan jumlah *trainable parameter* yang harus dipelajari menjadi sangat besar

```

class LocallyConnected2D:
    def __init__(self, kernel_weights, bias_weights, kernel_size,
stride=1):
        self.kernel = kernel_weights
        self.bias = bias_weights
        self.kH, self.kW = kernel_size
        self.stride = stride

    def forward(self, input_tensor):
        H, W, C_in = input_tensor.shape
        out_H = int((H - self.kH) / self.stride) + 1
        out_W = int((W - self.kW) / self.stride) + 1

        num_positions, patch_dim, C_out = self.kernel.shape

        output_tensor = np.zeros((out_H, out_W, C_out))

        for h in range(out_H):
            for w in range(out_W):
                position = h * out_W + w

                # batas patch pada gambar
                h_start = h * self.stride
                h_end = h_start + self.kH
                w_start = w * self.stride
                w_end = w_start + self.kW

                patch = input_tensor[h_start:h_end, w_start:w_end, :]
                patch_flatten = patch.flatten()

                kernel_position = self.kernel[position]
                bias_position = self.bias[position]

                output_tensor[h, w, :] = np.dot(patch_flatten,
kernel_position) + bias_position

        return output_tensor

```

2.2.8 Kelas CNN

Pada implementasi tugas besar ini, seluruh layer akan di-*wrap* dalam satu kelas bernama CNN. Dalam inisialisasi model, pengguna dapat menentukan ingin menggunakan layer apa saja dan bagaimana konfigurasinya dengan cara *stacking* layer tersebut. Bobot juga bisa disimpan atau di-*load* untuk mempermudah proses pembelajaran berikutnya.

```

class CNN:
    def __init__(self, layers):
        # list of layers
        self.layers = layers

```

```

def forward(self, x):
    for layer in self.layers:
        x = layer.forward(x)
    return x

def save_weights(self, filepath):
    weights_dict = {}
    for idx, layer in enumerate(self.layers):
        if hasattr(layer, 'kernel') and hasattr(layer, 'bias'):
            weights_dict[f'layer_{idx}_kernel'] = layer.kernel
            weights_dict[f'layer_{idx}_bias'] = layer.bias

    filepath = 'src/weights/' + filepath if not
    filepath.endswith('.npz') else filepath
    np.savez(filepath, **weights_dict)
    print(f"Bobot model berhasil disimpan ke {filepath}")

def load_weights(self, filepath):
    data = np.load(filepath)
    for idx, layer in enumerate(self.layers):
        kernel_key = f'layer_{idx}_kernel'
        bias_key = f'layer_{idx}_bias'

        if kernel_key in data.files and bias_key in data.files:
            layer.kernel = data[kernel_key]
            layer.bias = data[bias_key]

    print(f"Bobot model berhasil di-load dari {filepath}")

```

2.3 Forward Propagation RNN

Forward Propagation pada RNN merupakan proses mengalirkan data sekuensial berupa token *caption* dari *input layer* melewati *hidden layer* secara berulang hingga menghasilkan prediksi kata berikutnya. Pada *pipeline image captioning*, *Simple RNN* digunakan sebagai *decoder* yang bertugas menghasilkan deskripsi gambar berdasarkan *feature* visual yang dihasilkan oleh *CNN encoder*. Berbeda dengan CNN yang memproses data spasial, RNN memproses data secara berurutan (*sequence*) sehingga *output* pada *timestep* tertentu dipengaruhi oleh *hidden state* sebelumnya.

Pada implementasi *Simple RNN*, arsitektur yang digunakan adalah *pre-inject image captioning*, yaitu *feature vector* hasil CNN terlebih dahulu diproyeksikan menjadi *embedding* dan dijadikan *timestep* pertama sebelum token *caption* dimasukkan ke dalam *recurrent layer*. Dengan cara ini, informasi visual gambar dapat digunakan sebagai konteks awal dalam proses pembentukan *caption*.

2.3.1. EmbeddingScratch

Lapisan *embedding* berfungsi mengubah token *caption* berbentuk indeks integer menjadi representasi vektor *dense*. *Embedding layer* bekerja dengan mengambil baris *embedding* berdasarkan indeks token *input* sehingga setiap kata memiliki representasi numerik yang dapat diproses oleh RNN. Hasil dari proses *embedding* berupa tensor dengan dimensi (batch, seq_len, embed_dim).

```
class EmbeddingScratch:
    def __init__(self, weights: np.ndarray):
        self.W = weights.astype(np.float32)

    def forward(self, token_ids: np.ndarray) -> np.ndarray:
        token_ids = token_ids.astype(np.int64)
        return self.W[token_ids]
```

2.3.2. DenseScratch

Lapisan *dense* digunakan untuk melakukan transformasi linear terhadap *input* menggunakan operasi *dot product* antara *input* dan bobot kernel kemudian ditambahkan bias. Pada implementasi RNN ini, *dense layer* digunakan untuk dua keperluan utama, yaitu memproyeksikan *feature vector* CNN ke *embedding dimension* dan menghasilkan distribusi probabilitas *vocabulary* pada *output layer* menggunakan fungsi aktivasi *softmax*.

```
class DenseScratch:
    def __init__(self, W: np.ndarray, b: np.ndarray, activation: str |
None = None):
        self.W = W.astype(np.float32)
        self.b = b.astype(np.float32)
        self.activation = activation

    def forward(self, x: np.ndarray) -> np.ndarray:
        y = np.matmul(x, self.W) + self.b
        return apply_activation(y, self.activation)
```

2.3.3. SimpleRNNCellScratch

Kelas SimpleRNNCellScratch merupakan unit dasar *recurrent neural network* yang bertugas menghitung *hidden state* pada setiap *timestep*. Pada setiap langkah waktu, *hidden state* baru dihitung berdasarkan *input* saat ini dan *hidden state* sebelumnya. Fungsi aktivasi yang digunakan pada implementasi ini adalah fungsi tanh untuk memberikan non-linieritas pada model.

Persamaan *forward propagation* pada Simple RNN ditunjukkan sebagai berikut:

$$h_t = \tanh(x_t W_x + h_{t-1} W_h + b)$$

dengan:

- h_t = *hidden state* pada *timestep* ke- t
- x_t = *input* pada *timestep* ke- t
- W_x = bobot *input*
- W_h = bobot *reccurent*
- b = bias

```
class SimpleRNNCellScratch:
    def __init__(self, kernel: np.ndarray, recurrent_kernel:
np.ndarray, bias: np.ndarray,
                activation: str = "tanh"):

        self.Wx = kernel.astype(np.float32)
        self.Wh = recurrent_kernel.astype(np.float32)
        self.b = bias.astype(np.float32)
        self.activation = activation
        self.units = self.b.shape[0]

    def step(self, x_t: np.ndarray, h_prev: np.ndarray) -> np.ndarray:
        h = x_t @ self.Wx + h_prev @ self.Wh + self.b
        return apply_activation(h, self.activation)
```

2.3.4. SimpleRNNLayerScratch

Kelas SimpleRNNLayerScratch digunakan untuk memproses seluruh *sequence caption* secara berurutan menggunakan SimpleRNNCellScratch. Pada awal *forward propagation*, *hidden state* diinisialisasi menggunakan vektor nol. Selanjutnya, setiap *timestep* diproses satu per satu menggunakan *recurrent cell* dan hasil *hidden state* disimpan ke dalam *list outputs*. Pada akhir proses, seluruh *output sequence* digabung menggunakan `np.stack()`.

```
class SimpleRNNLayerScratch:
    def __init__(self, kernel: np.ndarray, recurrent_kernel:
np.ndarray, bias: np.ndarray,
                activation: str = "tanh", return_sequences: bool =
True):
        self.cell = SimpleRNNCellScratch(kernel, recurrent_kernel,
bias, activation)
        self.return_sequences = return_sequences
        self.units = self.cell.units

    def forward(self, x: np.ndarray, h0: np.ndarray | None = None):
```

```

batch_size, seq_len, _ = x.shape
if h0 is None:
    h_t = np.zeros((batch_size, self.units), dtype=np.float32)
else:
    h_t = h0.astype(np.float32)

outputs = []
for t in range(seq_len):
    h_t = self.cell.step(x[:, t, :], h_t)
    outputs.append(h_t)

output_seq = np.stack(outputs, axis=1)
if self.return_sequences:
    return output_seq, h_t
return h_t, h_t

```

2.3.5. RNNCaptionDecoderScratch

Kelas RNNCaptionDecoderScratch merupakan *wrapper* utama *decoder image captioning* berbasis Simple RNN. Pada implementasi ini digunakan metode *pre-inject* di mana *feature* gambar hasil CNN terlebih dahulu diproyeksikan menggunakan *dense layer* kemudian dijadikan *timestep* pertama sebelum *embedding* token *caption* dimasukkan ke *recurrent layer*.

Forward propagation dimulai dengan membentuk *sequence* gabungan antara *feature* gambar dan *embedding caption*. *Sequence* tersebut kemudian diproses oleh satu atau beberapa *layer* Simple RNN. *Output hidden state* pada setiap *timestep* selanjutnya diproyeksikan menggunakan *dense layer* dengan fungsi aktivasi *softmax* untuk menghasilkan distribusi probabilitas *vocabulary*.

Persamaan *output layer* ditunjukkan sebagai berikut:

$$y_t = \text{softmax}(h_t W_y + b_y)$$

Kata dengan probabilitas terbesar dipilih sebagai prediksi kata berikutnya hingga model menghasilkan token <end> atau mencapai panjang maksimum *caption*.

```

class RNNCaptionDecoderScratch:
    def __init__(self, feature_projection: DenseScratch, embedding:
EmbeddingScratch,
                rnn_layers: list[SimpleRNNLayerScratch],
output_dense: DenseScratch,
                idx_to_word: dict[int, str] | None = None,
                word_to_idx: dict[str, int] | None = None):
        self.feature_projection = feature_projection
        self.embedding = embedding

```

```

        self.rnn_layers = rnn_layers
        self.output_dense = output_dense
        self.idx_to_word = idx_to_word
        self.word_to_idx = word_to_idx

    def _make_preinject_sequence(self, image_features: np.ndarray,
                                input_token_ids: np.ndarray) -> np.ndarray:
        feature_embed =
self.feature_projection.forward(image_features)
        feature_embed = feature_embed[:, None, :]

        word_embed = self.embedding.forward(input_token_ids)
        x = np.concatenate([feature_embed, word_embed], axis=1)
        return x

    def forward_teacher_forcing(self, image_features: np.ndarray,
                                input_token_ids: np.ndarray) -> np.ndarray:
        x = self._make_preinject_sequence(image_features,
input_token_ids)

        # pass ke recurrent stack
        for rnn in self.rnn_layers:
            x, _ = rnn.forward(x)

        logits = self.output_dense.forward(x)
        return logits

    def predict_next_distribution(self, image_features: np.ndarray,
                                input_token_ids: np.ndarray) -> np.ndarray:
        probs = self.forward_teacher_forcing(image_features,
input_token_ids)
        return probs[:, -1, :] # (batch, vocab_size)

    def generate_greedy(self, image_feature: np.ndarray, start_id:
int, end_id: int,
                        max_len: int = 30, pad_id: int | None = None)
-> list[int]:
        if image_feature.ndim == 1:
            image_feature = image_feature[None, :]

        tokens = [start_id]
        generated = []
        for _ in range(max_len):
            inp = np.array(tokens, dtype=np.int64)[None, :]
            next_prob = self.predict_next_distribution(image_feature,
inp)[0]

            if pad_id is not None:
                next_prob[pad_id] = -1.0
            next_id = int(np.argmax(next_prob))

            if next_id == end_id:
                break

```

```

        generated.append(next_id)
        tokens.append(next_id)

    return generated

    def generate_caption(self, image_feature: np.ndarray, start_id:
int, end_id: int,
                        max_len: int = 30, pad_id: int | None = None)
-> str:
    ids = self.generate_greedy(image_feature, start_id, end_id,
max_len, pad_id)
    if self.idx_to_word is None:
        return " ".join(map(str, ids))
    return " ".join(self.idx_to_word.get(i, "<unk>") for i in ids)

```

2.4 Forward Propagation LSTM

Forward Propagation pada LSTM merupakan proses mengalirkan data sekuensial berupa token caption dari input layer melewati beberapa hidden layer hingga menghasilkan prediksi kata berikutnya. Pada pipeline image captioning, LSTM digunakan sebagai decoder yang bertugas menghasilkan deskripsi gambar berdasarkan fitur visual yang dihasilkan oleh CNN encoder.

Berbeda dengan Simple RNN yang hanya memiliki hidden state, LSTM memiliki komponen cell state yang berfungsi sebagai memori jangka panjang. Selain itu, LSTM juga memiliki mekanisme gate untuk menentukan informasi mana yang perlu disimpan, diperbarui, maupun dilupakan guna mempertahankan konteks kalimat yang lebih baik pada sekuens yang panjang.

2.4.1. Kelas Embedding Layer

Lapisan embedding berfungsi untuk mengubah token kata berbentuk indeks integer menjadi representasi vektor dense. Cara kerja embedding layer ini adalah dengan mengambil baris embedding berdasarkan indeks token input. Hasil dari proses ini berupa matriks embedding yang akan menjadi input untuk layer LSTM berikutnya.

```

class EmbeddingLayer:
    def __init__(self, weights):
        self.weights = {
            "embedding": weights.astype(np.float32)
        }

    def forward(self, x):
        return self.weights["embedding"][x]

```


2.4.2. Kelas Dense Layer

Lapisan dense pada implementasi LSTM digunakan sebagai output layer yang bertugas mengubah hidden state menjadi distribusi probabilitas *vocabulary*. Pada proses image captioning, layer ini digunakan untuk menentukan kata berikutnya. Pada bagian forward propagation, operasi utama yang dilakukan adalah operasi linear menggunakan dot product antara input dan bobot kernel kemudian ditambahkan dengan bias ($xW + b$). Terakhir, diterapkan fungsi aktivasi terhadap hasil operasi linear tersebut.

```
class DenseLayer:
    def __init__(self, weights, activation=None):
        self.weights = {
            "kernel": weights[0].astype(np.float32),
            "bias": weights[1].astype(np.float32),
        }
        self.activation = activation

    def _softmax(self, x):
        x = x - np.max(x, axis=-1, keepdims=True)
        exp_x = np.exp(x)
        return exp_x / np.sum(exp_x, axis=-1, keepdims=True)

    def forward(self, x):
        z = np.dot(x, self.weights["kernel"]) + self.weights["bias"]
        if self.activation == "softmax":
            return self._softmax(z)
        if self.activation == "relu":
            return np.maximum(0, z)
        if self.activation == "tanh":
            return np.tanh(z)

        return z
```

2.4.3. Kelas LSTM Layer

Lapisan LSTM merupakan inti utama dari *decoder image captioning* yang bertugas mempelajari hubungan antar kata pada data sekuensial. Pada setiap timestep, layer ini menerima input embedding kata dan hidden state sebelumnya untuk menghasilkan hidden state baru. Berbeda dengan Simple RNN, LSTM memiliki mekanisme gate yang terdiri dari *forget gate*, *input gate*, kandidat *cell state*, dan *output gate*. Gate tersebut digunakan untuk mengatur aliran informasi pada cell state sehingga model mampu mempertahankan informasi penting dalam jangka panjang.

Forward propagation pada lapisan ini dimulai dengan memastikan bentuk input memiliki dimensi batch. Setelah itu, hidden state (h) dan cell state (c) diinisialisasi dengan nilai

nol. Pada setiap timestep, embedding input saat ini akan diproses bersama hidden state sebelumnya melalui operasi linear $x(t)W_x + h(t-1)W_h + b$ untuk menghasilkan empat gate sekaligus, yaitu input gate, forget gate, candidate cell state, dan output gate.

Input gate menentukan informasi baru yang akan disimpan, *forget gate* menentukan informasi lama yang dipertahankan, kandidat cell state membentuk kandidat informasi baru, sedangkan output gate menentukan hidden state yang akan menjadi output *timestep* tersebut. Cell state kemudian diperbarui menggunakan kombinasi forget gate dan input gate, lalu hidden state timestep tersebut dihitung menggunakan output gate. Hidden state pada setiap timestep disimpan ke dalam list outputs dan pada akhir proses seluruh output digabung menjadi tensor sekuens menggunakan `np.stack()`.

```
class LSTMLayer:
    def __init__(self, units, return_sequences=True, weights=None):
        self.units = units
        self.return_sequences = return_sequences
        self.weights = {}

        if weights is not None:
            self.weights["kernel"] = weights[0].astype(np.float32)
            self.weights["recurrent_kernel"] =
weights[1].astype(np.float32)
            self.weights["bias"] = weights[2].astype(np.float32)

    def _sigmoid(self, x):
        return 1.0 / (1.0 + np.exp(-np.clip(x, -500, 500)))

    def _tanh(self, x):
        return np.tanh(np.clip(x, -500, 500))

    def forward(self, x):
        single_input = False

        if x.ndim == 2:
            x = np.expand_dims(x, axis=0)
            single_input = True

        batch_size, seq_len, input_dim = x.shape

        h = np.zeros((batch_size, self.units), dtype=np.float32)
        c = np.zeros((batch_size, self.units), dtype=np.float32)

        outputs = []

        for t in range(seq_len):
            #  $x(t)W_x + h(t-1)W_h + b$ 
            gates = (
                np.dot(x[:, t, :], self.weights["kernel"])
                + np.dot(h, self.weights["recurrent_kernel"])
                + self.weights["bias"]
            )
```

```

        )

        i_gate = self._sigmoid(gates[:, :self.units])
        f_gate = self._sigmoid(gates[:, self.units:2 *
self.units])
        c_tilde = self._tanh(gates[:, 2 * self.units:3 *
self.units])
        o_gate = self._sigmoid(gates[:, 3 * self.units:])

        c = f_gate * c + i_gate * c_tilde
        h = o_gate * self._tanh(c)

        outputs.append(h.copy())
        outputs = np.stack(outputs, axis=1)

        if not self.return_sequences:
            outputs = h

        if single_input:
            outputs = outputs[0]

        return outputs

```

2.4.4. Kelas LSTMFromScratch

Kelas LSTMFromScratch merupakan *wrapper class* yang bertugas mengintegrasikan *embedding layer*, *feature projection layer*, layer LSTM, dan output layer menjadi satu pipeline. Pada tahap awal, fitur gambar hasil CNN encoder akan diproyeksikan menjadi embedding menggunakan feature projection layer. Setelah itu, token caption diubah menjadi embedding kata menggunakan embedding layer. Kedua embedding tersebut kemudian digabung menjadi satu sekuens dan diproses secara berurutan oleh layer-layer LSTM. Pada layer terakhir, output hidden state diteruskan ke dense output layer untuk menghasilkan probabilitas vocabulary pada setiap timestep.

```

class LSTMFromScratch:
    def __init__(
        self,
        embedding_layer,
        feature_projection_layer,
        lstm_layers,
        output_layer,
        word_to_idx,
        idx_to_word,
        max_len=30,
    ):
        self.embedding_layer = embedding_layer
        self.feature_projection_layer = feature_projection_layer
        self.lstm_layers = lstm_layers

```

```

        self.output_layer = output_layer

        self.word_to_idx = word_to_idx
        self.idx_to_word = idx_to_word
        self.max_len = max_len

        self.start_id = word_to_idx["<start>"]
        self.end_id = word_to_idx["<end>"]

    def forward(self, cnn_feature, input_tokens):
        cnn_feature = cnn_feature.astype(np.float32)
        image_embedding =
self.feature_projection_layer.forward(cnn_feature)
        image_embedding = image_embedding.reshape(1, -1)

        word_embeddings = self.embedding_layer.forward(input_tokens)
        sequence = np.concatenate(
            [image_embedding, word_embeddings],
            axis=0,
        )

        output = sequence
        for lstm_layer in self.lstm_layers:
            output = lstm_layer.forward(output)

        probabilities = self.output_layer.forward(output)
        return probabilities

    def greedy_decode(self, cnn_feature):
        token_ids = [self.start_id]
        caption_words = []

        for _ in range(self.max_len):
            input_tokens = np.array(token_ids, dtype=np.int64)
            probabilities = self.forward(cnn_feature, input_tokens)
            next_id = int(np.argmax(probabilities[-1]))

            if next_id == self.end_id:
                break

            token_ids.append(next_id)
            word = self.idx_to_word[str(next_id)]
            caption_words.append(word)
        return " ".join(caption_words)

```

BAB 3

HASIL PENGUJIAN

3.1 Hasil Pengujian CNN

3.1.1 Perbandingan shared vs non-shared parameter

Pada pengujian perbandingan antara *shared* dan *non-shared* parameter, digunakan arsitektur yang lebih sederhana karena terjadi *bottleneck* saat proses *training*. Ketika digunakan arsitektur yang sama dengan model kombinasi 12 yang terdiri dari 3 layer, filter berukuran 32-64-128, kernel berukuran 5 x 5, dan *average pooling*, proses *training* tidak mengalami progres apapun dalam kurun waktu satu jam, bahkan satu epoch pun tidak selesai. Oleh karena itu, arsitektur model dibuat lebih sederhana agar waktu *training* tidak terlalu lama. Konfigurasi yang digunakan untuk pengujian ini adalah terdiri dari 2 layer, filter berukuran 4-8, kernel berukuran 3 x 3, dan *average pooling*.

Model Non-Shared Parameter		
Macro-F1-Score : 0.6714		
<pre>model_12_non_shared = keras.Sequential([keras.layers.InputLayer(input_shape=(150, 150, 3)), keras.layers.AveragePooling2D(pool_size=(4, 4), strides=4), keras.layers.ZeroPadding2D(padding=(2, 2)), keras.layers.LocallyConnected2D(4, (3, 3), activation='relu'), keras.layers.AveragePooling2D(pool_size=(2, 2), strides=2), keras.layers.ZeroPadding2D(padding=(2, 2)), keras.layers.LocallyConnected2D(8, (3, 3), activation='relu'), keras.layers.AveragePooling2D(pool_size=(2, 2), strides=2), keras.layers.Flatten(), keras.layers.Dense(128, activation='relu'), keras.layers.Dense(6, activation='softmax')])</pre>		
Layer (type)	Output Shape	Param #
=====	=====	=====
average_pooling2d (Average Pooling2D)	(None, 37, 37, 3)	0
zero_padding2d (ZeroPadding2D)	(None, 41, 41, 3)	0
locally_connected2d (Local	(None, 39, 39, 4)	170352

```

lyConnected2D)

average_pooling2d_1 (AveragePooling2D) (None, 19, 19, 4) 0

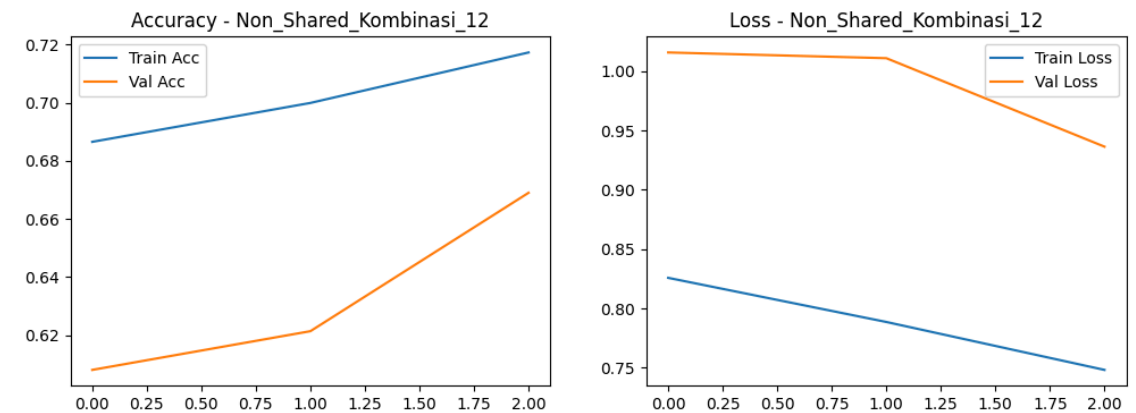
zero_padding2d_1 (ZeroPadding2D) (None, 23, 23, 4) 0

locally_connected2d_1 (LocallyConnected2D) (None, 21, 21, 8) 130536

average_pooling2d_2 (AveragePooling2D) (None, 10, 10, 8) 0

...
Total params: 404190 (1.54 MB)
Trainable params: 404190 (1.54 MB)
Non-trainable params: 0 (0.00 Byte)

```



Model Shared Parameter

Macro-F1-Score : 0.6772

```

model_12_shared = keras.Sequential([
    keras.layers.InputLayer(input_shape=(150, 150, 3)),

    keras.layers.AveragePooling2D(pool_size=(4, 4), strides=4),

    keras.layers.ZeroPadding2D(padding=(2, 2)),
    keras.layers.Conv2D(4, (3, 3), activation='relu'),
    keras.layers.AveragePooling2D(pool_size=(2, 2), strides=2),

    keras.layers.ZeroPadding2D(padding=(2, 2)),
    keras.layers.Conv2D(8, (3, 3), activation='relu'),
    keras.layers.AveragePooling2D(pool_size=(2, 2), strides=2),

```

```

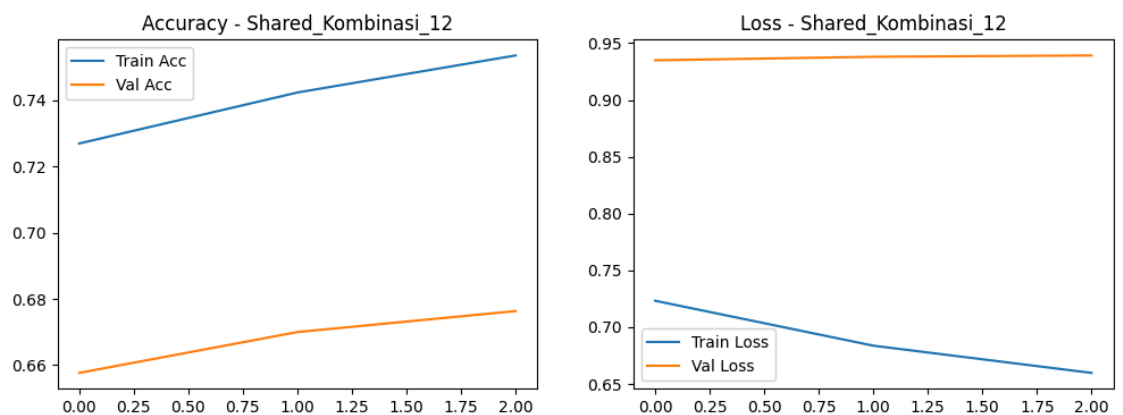
keras.layers.Flatten(),

keras.layers.Dense(32, activation='relu'),
keras.layers.Dense(6, activation='softmax')
])

```

Layer (type)	Output Shape	Param #
average_pooling2d_3 (AveragePooling2D)	(None, 37, 37, 3)	0
zero_padding2d_2 (ZeroPadding2D)	(None, 41, 41, 3)	0
conv2d (Conv2D)	(None, 39, 39, 4)	112
average_pooling2d_4 (AveragePooling2D)	(None, 19, 19, 4)	0
zero_padding2d_3 (ZeroPadding2D)	(None, 23, 23, 4)	0
conv2d_1 (Conv2D)	(None, 21, 21, 8)	296
average_pooling2d_5 (AveragePooling2D)	(None, 10, 10, 8)	0
flatten_1 (Flatten)	(None, 800)	0

...
Total params: 26238 (102.49 KB)
Trainable params: 26238 (102.49 KB)
Non-trainable params: 0 (0.00 Byte)



Berdasarkan hasil pengujian, terlihat bahwa Macro F-1 score untuk *shared* parameter (0.6772) lebih tinggi dibandingkan untuk *non-shared* (0.6714) meskipun jumlah trainable parameternya jauh lebih sedikit. Jumlah parameter untuk *non-shared* adalah sebanyak 404190, sedangkan untuk *shared* sebanyak 26238, yang artinya jumlah parameter pada *non-shared* untuk kasus ini 15 kali lipat lebih banyak dibandingkan dengan *shared*. Hal ini menandakan bahwa jumlah parameter lebih banyak tidak menjamin performa model lebih baik. Selain waktu *training* yang jauh lebih lama, model *non-shared* parameter juga memiliki kecenderungan mengalami *overfitting* karena kernel yang digunakan untuk ekstraksi fitur berbeda pada setiap pergeseran sehingga model malah menghafal pola tersebut.

Dari hasil pengujian ini, terbukti bahwa hipotesis performa *non-shared* parameter sangat buruk dan tidak efektif. Oleh karena itu, arsitektur *non-shared* parameter sudah sangat jarang digunakan karena waktu *training* yang lama dan hasil yang lebih buruk. Dengan arsitektur serupa namun menggunakan *shared parameter* terbukti menghasilkan nilai Macro F-1 score yang lebih baik dan proses training yang jauh lebih cepat.

3.1.2 Pengaruh jumlah layer konvolusi

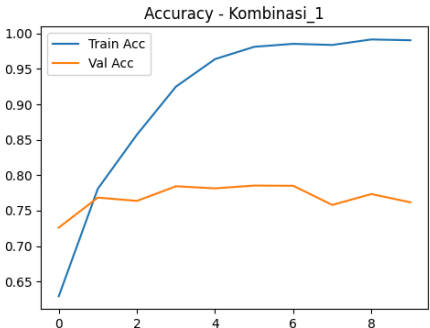
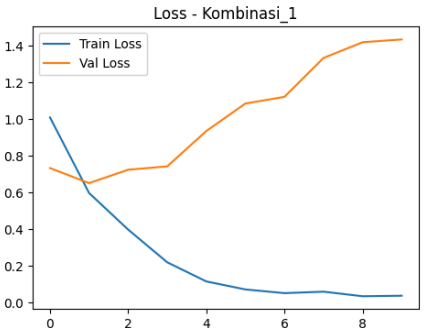
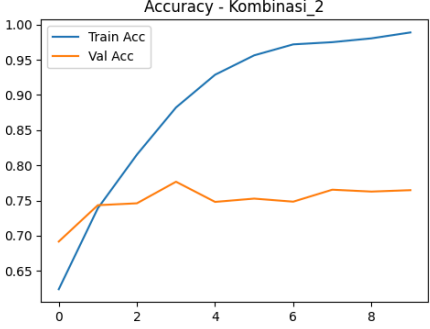
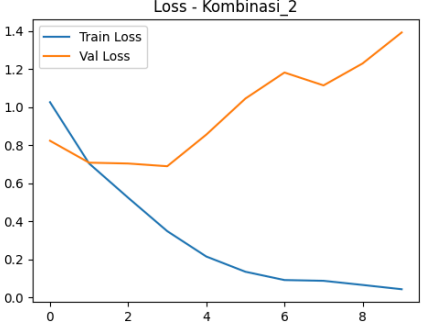
Pada pengujian dengan 16 arsitektur, variasi *hyperparameter* yang digunakan mempertimbangkan variasi pengaruh jumlah layer konvolusi, banyak filter per layer konvolusi, ukuran filter per layer konvolusi, dan jenis *pooling* yang digunakan. Untuk mempermudah proses analisis, berikut merupakan pemetaan kombinasi yang digunakan untuk pengujian ini.

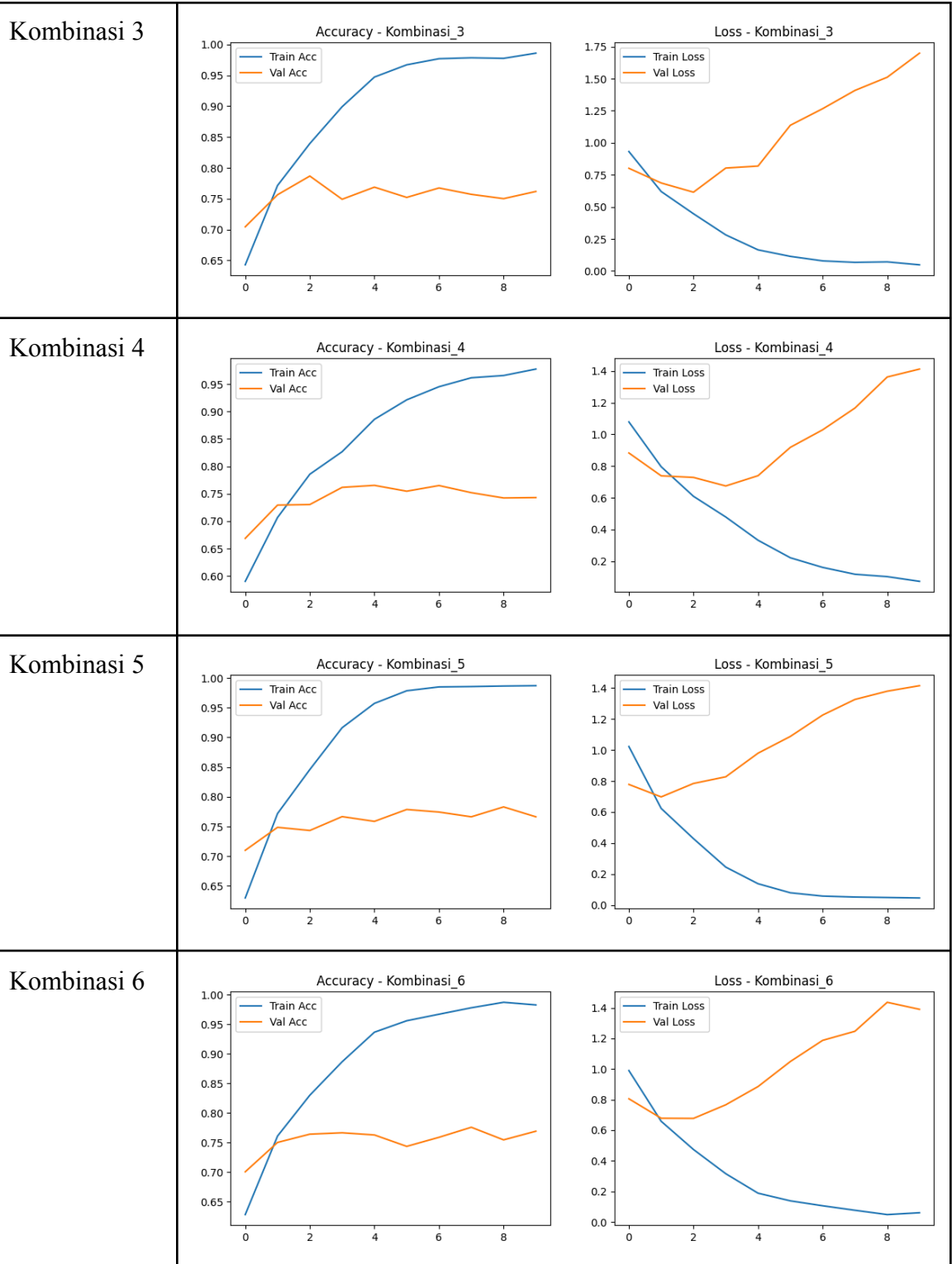
Tabel 3.1.2.1 Konfigurasi 16 Arsitektur

Nomor Kombinasi	Arsitektur	Macro F1 score
Kombinasi 1	2 layer, filter [32,64], kernel 3 x 3, <i>max pooling</i>	0.7598
Kombinasi 2	2 layer, filter [32,64], kernel 3 x 3, <i>avg pooling</i>	0.7640
Kombinasi 3	2 layer, filter [32,64], kernel 5 x 5, <i>max pooling</i>	0.7589
Kombinasi 4	2 layer, filter [32,64], kernel 5 x 5, <i>avg pooling</i>	0.7432
Kombinasi 5	2 layer, filter [64, 128], kernel 3 x 3, <i>max pooling</i>	0.7683
Kombinasi 6	2 layer, filter [64, 128], kernel 3 x 3, <i>avg pooling</i>	0.7699
Kombinasi 7	2 layer, filter [64, 128], kernel 5 x 5, <i>max pooling</i>	0.7424
Kombinasi 8	2 layer, filter [64, 128], kernel 5 x 5, <i>avg pooling</i>	0.7307

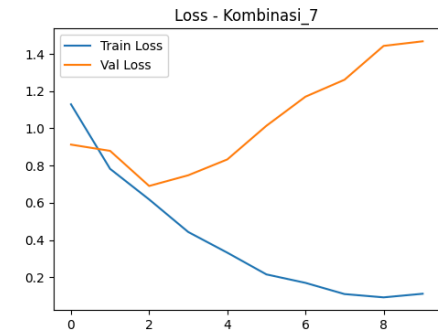
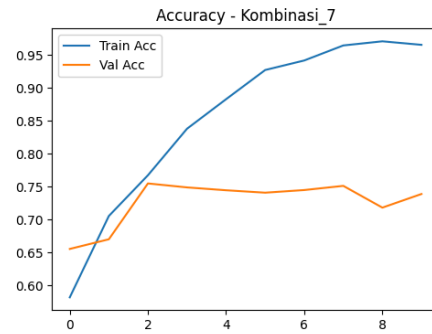
Kombinasi 9	3 layer, filter [32,64,128], kernel 3 x 3, <i>max pooling</i>	0.7983
Kombinasi 10	3 layer, filter [32,64,128], kernel 3 x 3, <i>avg pooling</i>	0.7832
Kombinasi 11	3 layer, filter [32,64,128], kernel 5 x 5, <i>max pooling</i>	0.7920
Kombinasi 12	3 layer, filter [32,64,128], kernel 5 x 5, <i>avg pooling</i>	0.8082
Kombinasi 13	3 layer, filter [64,128, 256], kernel 3 x 3, <i>max pooling</i>	0.7923
Kombinasi 14	3 layer, filter [64,128, 256], kernel 3 x 3, <i>avg pooling</i>	0.7910
Kombinasi 15	3 layer, filter [64,128, 256], kernel 5 x 5, <i>max pooling</i>	0.7735
Kombinasi 16	3 layer, filter [64,128, 256], kernel 5 x 5, <i>avg pooling</i>	0.7918

Tabel 3.1.2.2 Perbandingan *Loss Validation* dan *Train Validation*

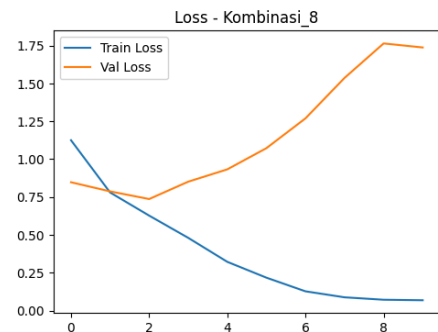
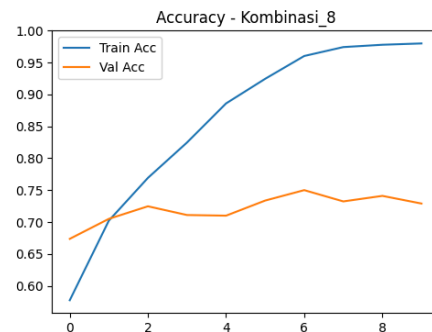
Nomor Kombinasi	Perbandingan <i>Loss Validation</i> dan <i>Train Validation</i>
Kombinasi 1	 
Kombinasi 2	 



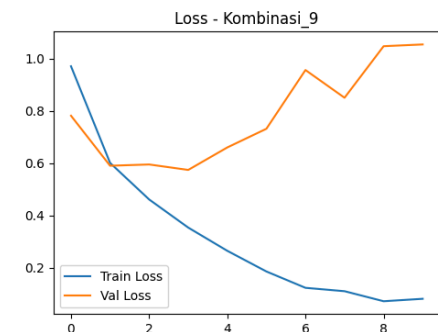
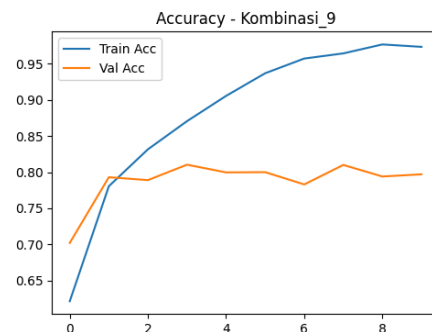
Kombinasi 7



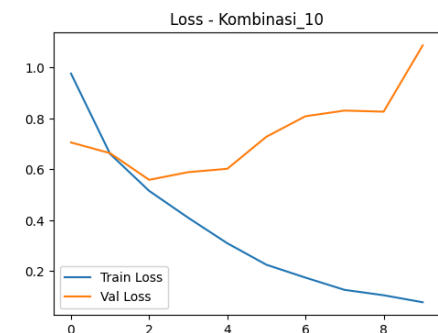
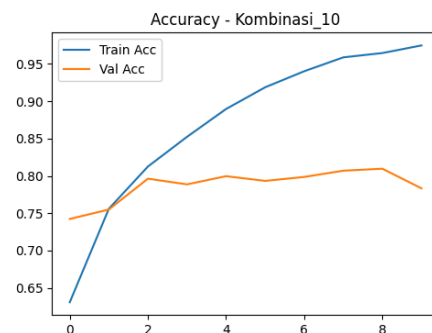
Kombinasi 8



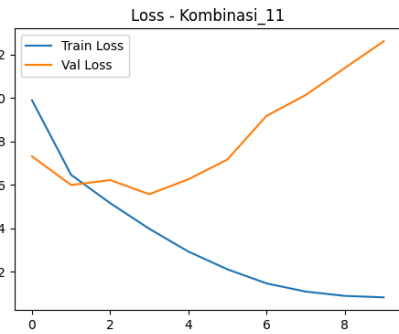
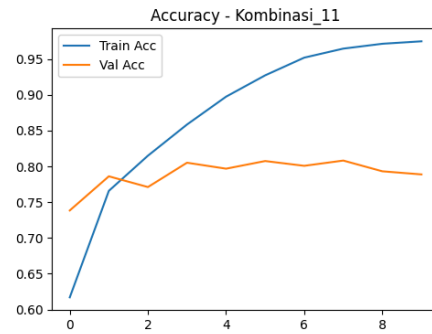
Kombinasi 9



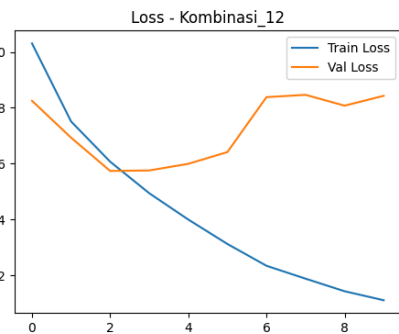
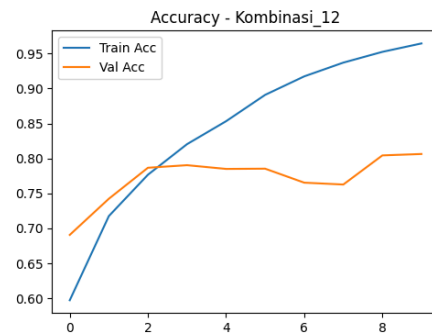
Kombinasi 10



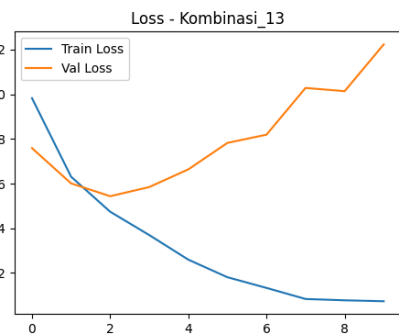
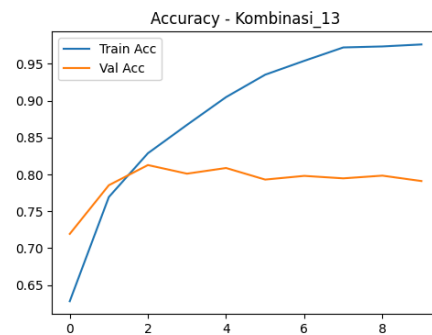
Kombinasi 11



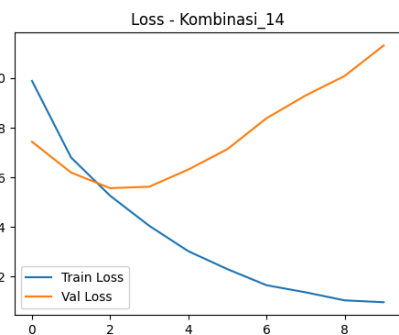
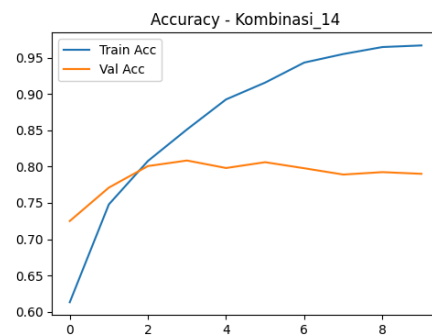
Kombinasi 12

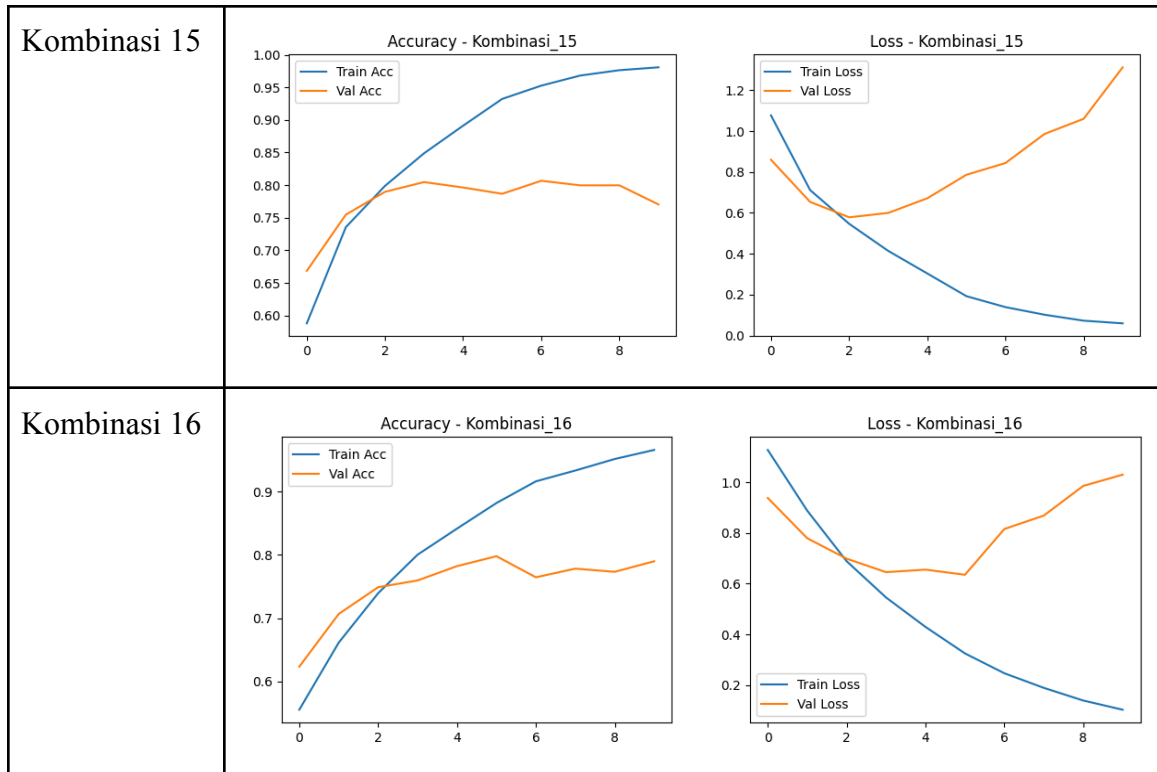


Kombinasi 13



Kombinasi 14





Tabel 3.1.2.2 Rekap Urutan Top Macro F1 Score

Rekap Urutan Top Macro F1 Score	
Kombinasi_12	: 0.8082
Kombinasi_9	: 0.7983
Kombinasi_13	: 0.7923
Kombinasi_11	: 0.7920
Kombinasi_16	: 0.7918
Kombinasi_14	: 0.7910
Kombinasi_10	: 0.7832
Kombinasi_15	: 0.7735
Kombinasi_6	: 0.7699
Kombinasi_5	: 0.7683
Kombinasi_2	: 0.7640
Kombinasi_1	: 0.7598
Kombinasi_3	: 0.7589
Kombinasi_4	: 0.7432
Kombinasi_7	: 0.7424
Kombinasi_8	: 0.7307

Berdasarkan hasil eksperimen, terlihat bahwa top 8 F1 score merupakan model dengan kombinasi yang menggunakan 3 layer (layer yang lebih banyak). Hal ini menandakan bahwa jumlah layer berbanding lurus terhadap F1 score, semakin banyak layer yang digunakan maka performanya akan semakin baik.

3.1.3 Pengaruh banyak filter per layer konvolusi

Merujuk pada tabel 3.1.2.1 dan 3.1.2.2, eksperimen pengaruh banyak filter per layer dapat diamati pada kombinasi 1,2,3,4 dengan 5,6,7,8 atau 9,10,11,12 dengan 13,14,15,16. Hasil F1 score pada kedua kelompok tersebut tidak terlihat polanya dengan jelas. Polanya selang-seling, terkadang filter yang lebih banyak akan menghasilkan F1 score lebih tinggi, namun terkadang filter yang lebih sedikit akan menghasilkan F1 score lebih tinggi juga. Namun, apabila digeneralisasi, penggunaan jumlah filter yang semakin banyak cenderung memberikan hasil yang lebih baik. Hal ini dikarenakan filter yang banyak pada layer akhir akan memungkinkan model mengenali pola yang lebih kompleks.

3.1.4 Pengaruh ukuran filter per layer konvolusi

Merujuk pada tabel 3.1.2.1 dan 3.1.2.2, eksperimen pengaruh ukuran filter salah satunya dapat diamati pada kombinasi 1,2 dengan 3,4. Berdasarkan F1 score, kombinasi 1,2 memiliki nilai yang lebih tinggi dibandingkan 3,4. Pola serupa juga terlihat pada 5,6 dengan 7,8 dimana nilai F1 score lebih tinggi pada kombinasi 5,6. Namun, pola tersebut tidak terlihat pada kombinasi 9,10 dengan 11,12 dan 13,14 dengan 15,16. Pada kasus ini, ukuran filter yang kecil memiliki kecenderungan menghasilkan performa yang lebih baik dibandingkan dengan yang berukuran lebih besar namun tidak berlaku untuk semua kombinasi. Hal ini berkaitan dengan semakin besar ukuran kernel maka konteks spasial yang dapat ditangkap akan menjadi semakin baik dan generalisasinya juga akan lebih baik. Akan tetapi, kasus untuk tugas besar ini lebih memiliki kecenderungan sebaliknya dan hal ini kemungkinan disebabkan oleh jenis dataset yang digunakan lebih cocok memakai kernel kecil untuk menangkap detail yang lebih tajam.

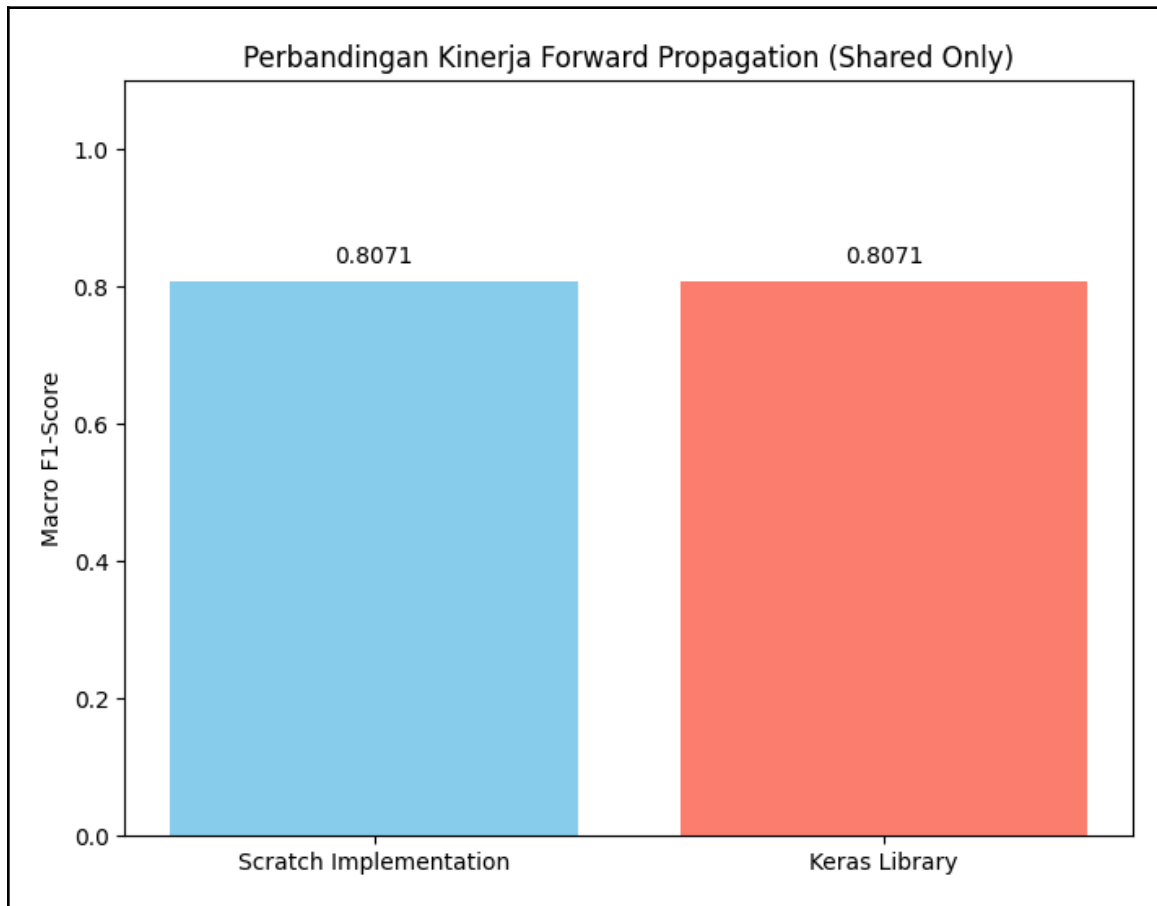
3.1.5 Pengaruh jenis pooling layer

Merujuk pada tabel 3.1.2.1 dan 3.1.2.2, eksperimen pengaruh jenis pooling menunjukkan bahwa *max pooling* secara umum memberikan hasil lebih baik dibandingkan *average pooling*. *Max pooling* mempertahankan fitur paling dominan dari hasil konvolusi sehingga informasi penting pada citra tidak mudah hilang. Hal ini terlihat dari banyaknya kombinasi yang menggunakan *max pooling* yang berada pada peringkat atas seperti kombinasi 9, 11, 13 dan pada kombinasi 3,4 serta 7,8.

3.1.6 Perbandingan Keras vs from scratch

Pada eksperimen ini, dilakukan perbandingan hasil prediksi antara model Keras dengan model *from scratch* yang telah kami implementasikan. Kedua model menggunakan bobot yang sama dari hasil *training* pada kombinasi 12 yang merupakan peroleh F1 score tertinggi. Dari hasil *forward propagation* tersebut, diperoleh bahwa F1 score dari model Keras dan model *from scratch* adalah sama di angka 0.8071. Hal ini membuktikan bahwa implementasi *from scratch* untuk *forward propagation* pada CNN berhasil dan sudah

berjalan dengan baik. Hasil prediksi selengkapnya dapat dilihat pada folder `src/cnn/hasil_prediksi.csv` di *repository*.



3.2 Hasil Pengujian Simple RNN dan LSTM

3.2.1. Perbandingan jumlah layer: RNN

Pengujian jumlah layer pada RNN menunjukkan bahwa penambahan layer tidak selalu menghasilkan performa yang lebih baik. Pada ukuran hidden state 128, terlihat pola bahwa model dengan 2 dan 3 layer memiliki performa lebih baik dibandingkan model dengan 1 layer. Sehingga, didapatkan kecenderungan bahwa semakin banyak layer yang digunakan, nilai BLEU-4 dan METEOR semakin meningkat.

Namun, pola ini tidak selalu berlaku. Pada ukuran hidden state 512, model justru memperoleh performa terbaik pada arsitektur 1 layer dan performa terendah pada arsitektur 2 layer. Hal ini menunjukkan bahwa pengaruh jumlah layer terhadap performa model sangat bergantung pada konfigurasi hidden state dan kemampuan model dalam melakukan generalisasi.

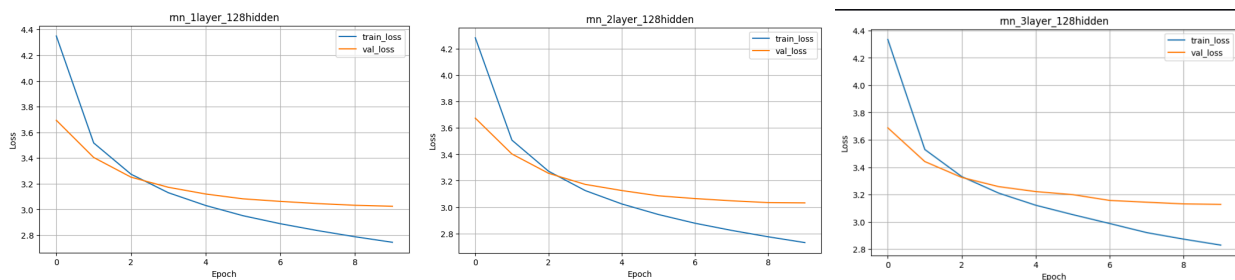
Selain itu, penambahan jumlah layer juga menyebabkan waktu training meningkat cukup signifikan karena kompleksitas model dan jumlah parameter yang harus diproses menjadi lebih besar.

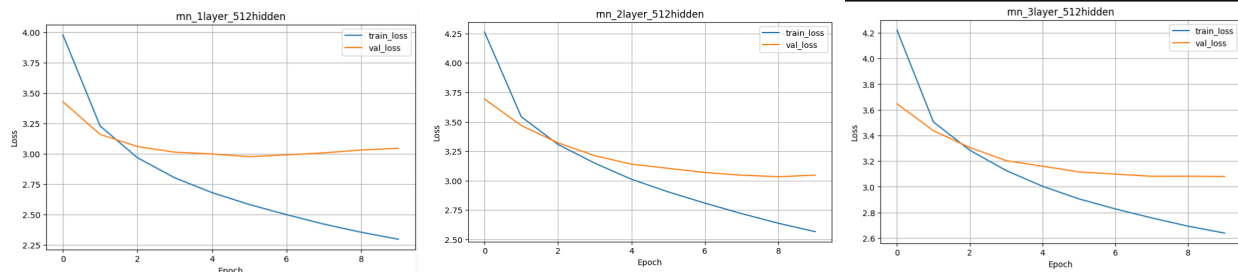
Variasi (128 hidden)	BLEU-4 score	METEOR score	train time	evaluation time
1 Layer	0.10369	0.292966	226.468763	1612.186594
2 Layer	0.108400	0.302030	238.344616	1842.950079
3 Layer	0.108164	0.293262	328.186380	778.080486

Variasi (512 hidden)	BLEU-4 score	METEOR score	train time	evaluation time
1 Layer	0.127567	0.324534	663.736187	1353.757508
2 Layer	0.116353	0.310350	971.932288	1579.042459
3 Layer	0.121389	0.313884	2110.041780	1353.757508

Meskipun pola performa model tidak konsisten, berdasarkan grafik loss, seluruh konfigurasi menunjukkan pola bahwa model dengan jumlah layer yang lebih sedikit dapat mencapai training loss yang lebih kecil.

Pada konfigurasi 128 hidden state, model yang memiliki jumlah layer lebih banyak cenderung mengalami overfitting yang lebih besar. Sebaliknya, pada konfigurasi 512 hidden state, model yang memiliki jumlah layer lebih sedikit cenderung mengalami overfitting yang lebih besar. Hal ini memperkuat pernyataan sebelumnya bahwa pengaruh jumlah layer terhadap performa model sangat bergantung pada konfigurasi yang digunakan.





3.2.2. Perbandingan jumlah layer: LSTM

Pengujian jumlah layer pada LSTM menunjukkan bahwa penambahan layer tidak selalu menghasilkan performa yang lebih baik. Pada ukuran hidden state 512, terlihat pola bahwa semakin banyak layer yang digunakan, nilai BLEU-4 dan METEOR semakin menurun.

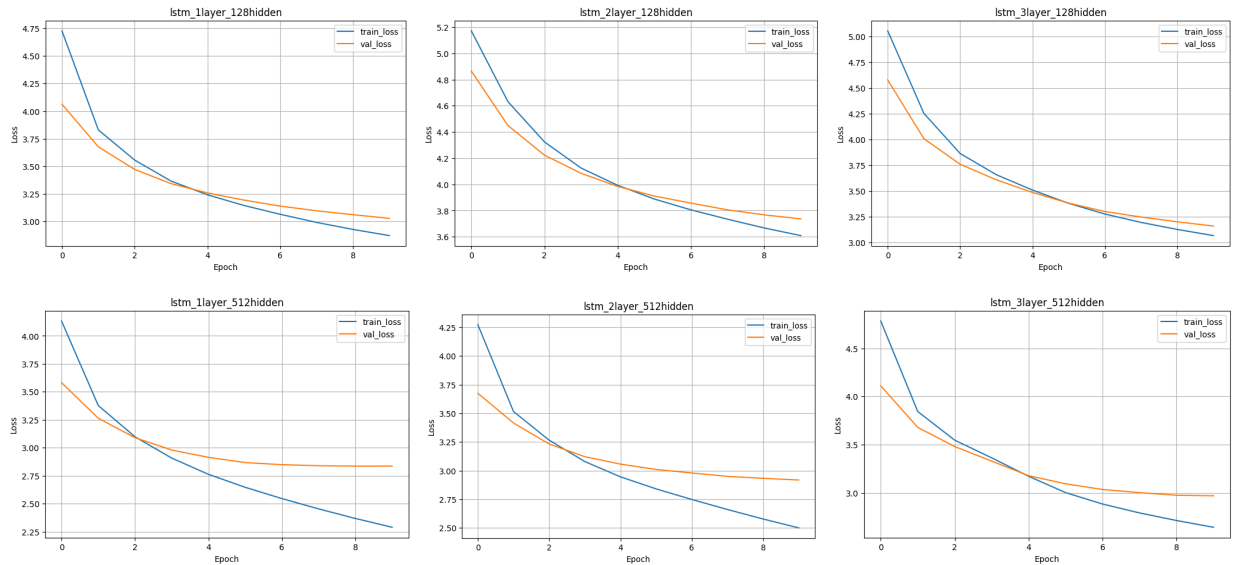
Namun, pola ini tidak selalu berlaku. Pada ukuran hidden state 128, model justru memperoleh performa terbaik pada arsitektur 2 layer dan performa terendah pada arsitektur 1 layer. Hal ini menunjukkan bahwa pengaruh jumlah layer terhadap performa model sangat bergantung pada konfigurasi hidden state dan kemampuan model dalam melakukan generalisasi.

Selain itu, penambahan jumlah layer juga menyebabkan waktu training meningkat cukup signifikan karena kompleksitas model dan jumlah parameter yang harus diproses menjadi lebih besar.

Variasi (128 hidden)	BLEU-4 score	METEOR score	train time	evaluation time
1 Layer	0.105577	0.299974	289.399894	676.961346
2 Layer	0.113183	0.301490	340.966963	698.441146
3 Layer	0.112747	0.299218	413.047829	726.970896

Variasi (512 hidden)	BLEU-4 score	METEOR score	train time	evaluation time
1 Layer	0.157926	0.357980	1031.337243	761.344919
2 Layer	0.142241	0.330821	1670.788568	825.040261
3 Layer	0.126351	0.325143	2261.770083	888.055025

Meskipun pola performa model tidak konsisten, berdasarkan grafik loss, seluruh konfigurasi menunjukkan pola bahwa model dengan jumlah layer yang lebih sedikit lebih mudah mengalami overfitting. Hal ini dapat dilihat dari gap antara training loss dan validation loss yang lebih besar pada model 1 layer dibandingkan model 2 layer dan 3 layer.



3.2.3. Perbandingan ukuran hidden state: RNN

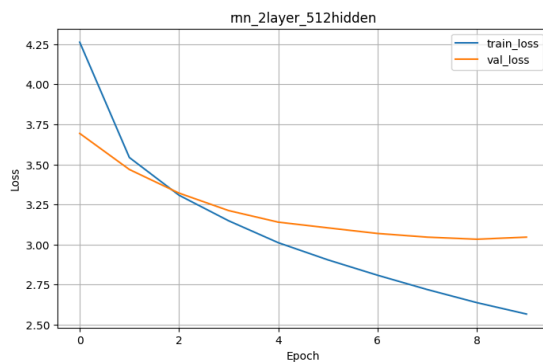
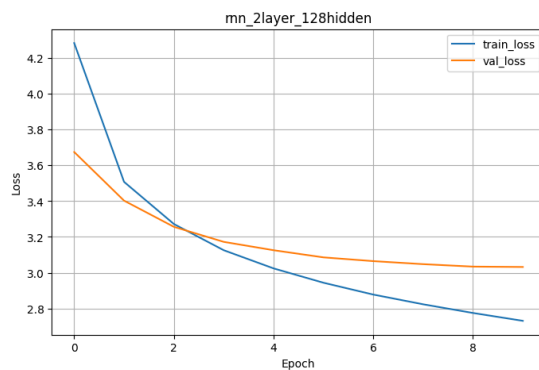
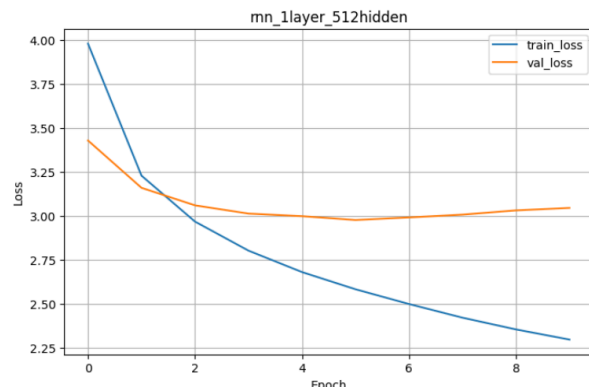
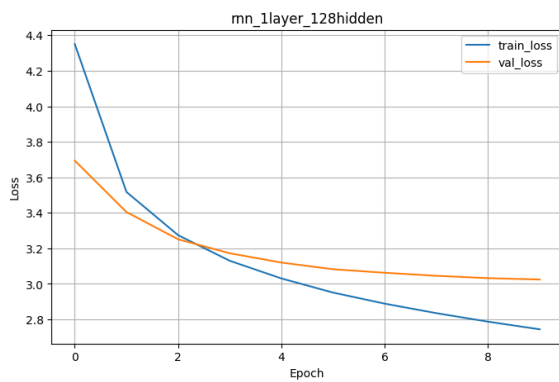
Pengujian ukuran hidden state pada RNN menunjukkan bahwa penggunaan hidden state yang lebih besar cenderung menghasilkan performa captioning yang lebih baik. Berdasarkan hasil evaluasi, semakin besar ukuran hidden state maka nilai BLEU-4 dan METEOR juga semakin meningkat. Namun, penggunaan hidden state yang lebih besar juga menyebabkan waktu training menjadi lebih lama karena jumlah parameter dan proses komputasi yang semakin besar.

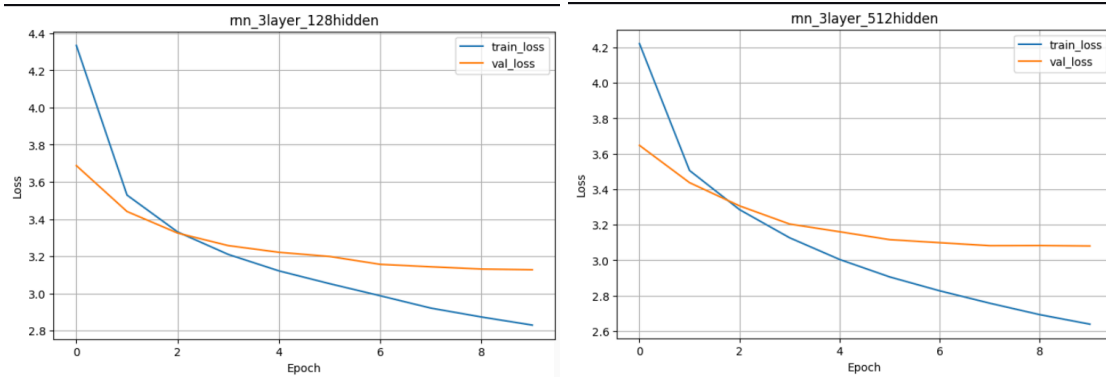
Variasi (1 Layer)	BLEU-4 score	METEOR score	train time	evaluation time
128 hidden	0.10369	0.292966	226.468763	1612.186594
512 hidden	0.127567	0.324534	663.736187	1353.757508

Variasi (2 Layer)	BLEU-4 score	METEOR score	train time	evaluation time
128 hidden	0.108400	0.302030	238.344616	1842.950079
512 hidden	0.116353	0.310350	971.932288	1579.042459

Variasi (3 Layer)	BLEU-4 score	METEOR score	train time	evaluation time
128 hidden	0.108164	0.293262	328.186380	778.080486
512 hidden	0.121389	0.313884	2110.041780	1353.757508

Berdasarkan grafik loss, model dengan ukuran hidden state yang lebih besar dapat mencapai training loss yang lebih rendah. Namun, model dengan ukuran hidden state yang lebih besar juga memiliki kecenderungan overfitting yang lebih tinggi. Hal ini terlihat dari gap antara training loss dan validation loss yang semakin besar.





3.2.4. Perbandingan ukuran hidden state: LSTM

Pengujian ukuran hidden state pada LSTM menunjukkan bahwa semakin besar ukuran hidden state, nilai BLEU-4 dan METEOR juga semakin meningkat. Hal ini menunjukkan bahwa penggunaan hidden state yang lebih besar cenderung menghasilkan performa captioning yang lebih baik. Namun, penggunaan hidden state yang lebih besar juga menyebabkan waktu training menjadi lebih lama karena jumlah parameter dan proses komputasi yang semakin besar.

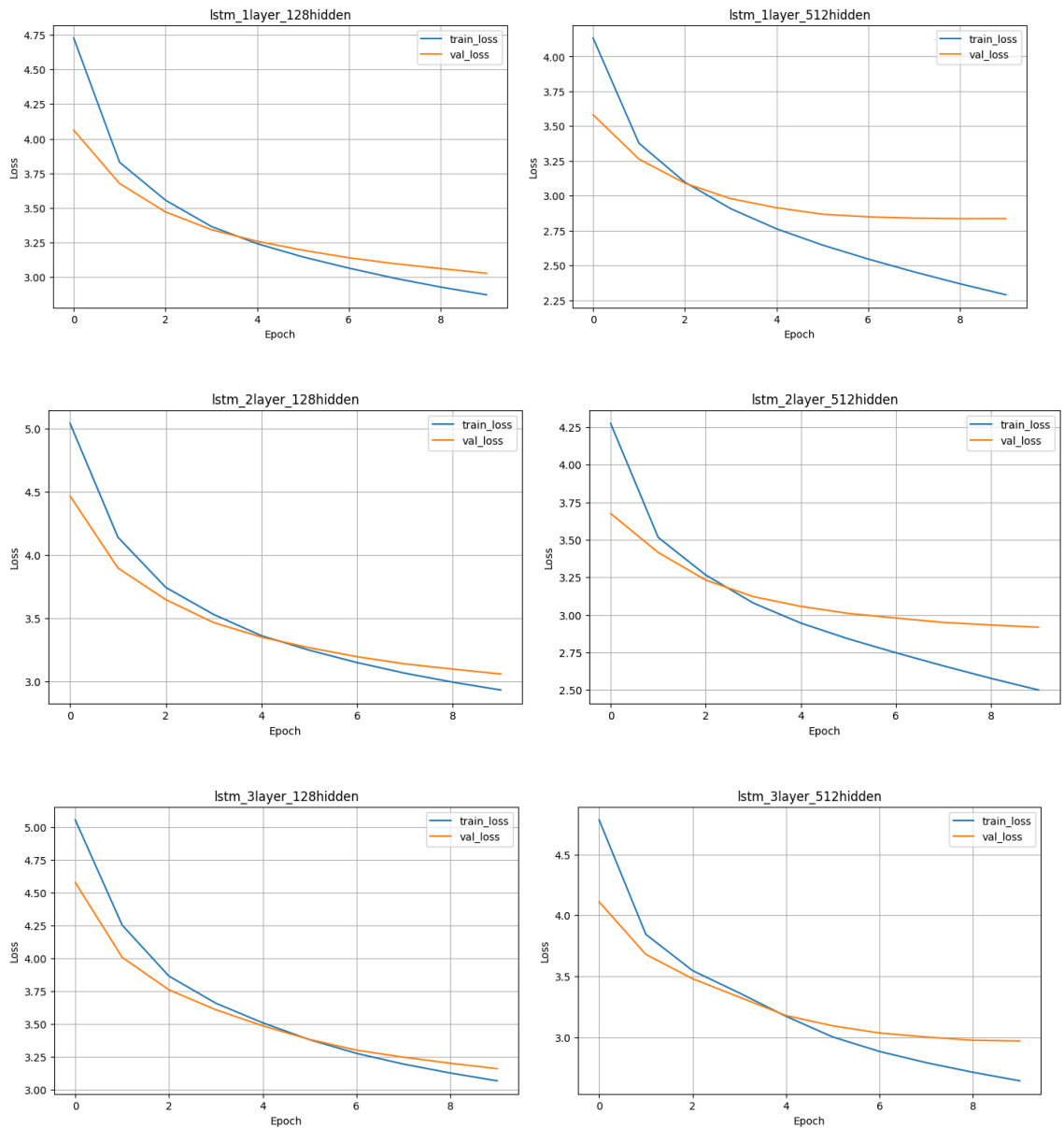
Variasi (1 Layer)	BLLEU-4 score	METEOR score	train time	evaluation time
128 hidden	0.105577	0.299974	289.399894	676.961346
512 hidden	0.157926	0.357980	1031.337243	761.344919

Variasi (2 Layer)	BLLEU-4 score	METEOR score	train time	evaluation time
128 hidden	0.113183	0.301490	340.966963	698.441146
512 hidden	0.142241	0.330821	1670.788568	825.040261

Variasi (3 Layer)	BLLEU-4 score	METEOR score	train time	evaluation time
128 hidden	0.112747	0.299218	413.047829	726.970896
512 hidden	0.126351	0.325143	2261.770083	888.055025

Berdasarkan grafik loss, model dengan ukuran hidden state yang lebih besar dapat mencapai training loss yang lebih rendah. Namun, model dengan ukuran hidden state

yang lebih besar juga memiliki kecenderungan overfitting yang lebih tinggi. Hal ini terlihat dari gap antara training loss dan validation loss yang semakin besar.



3.2.5. Perbandingan RNN vs LSTM

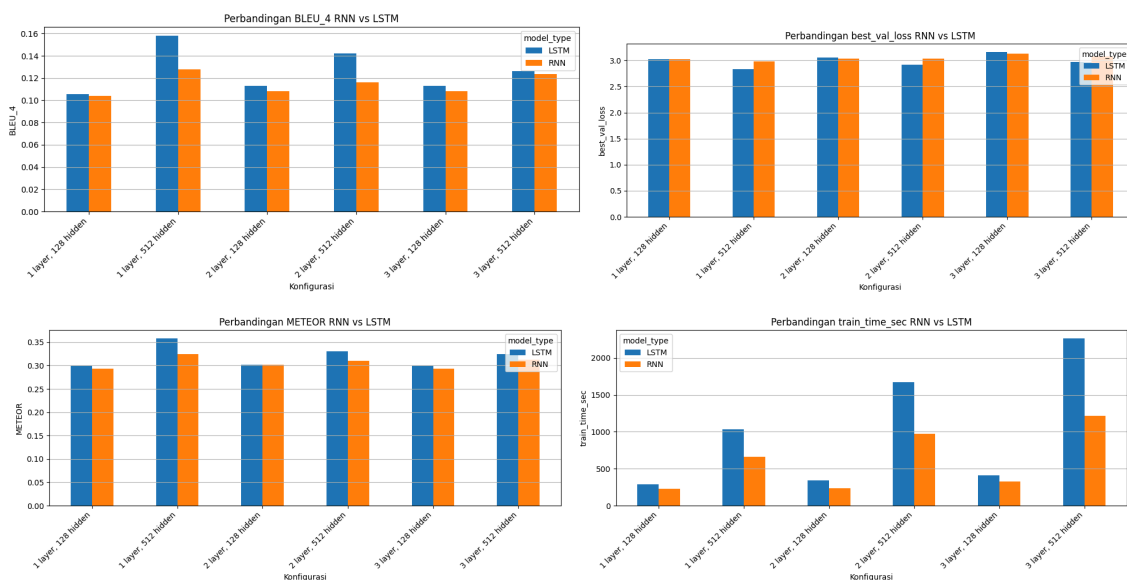
Pada pengujian ini, dilakukan perbandingan antara decoder Simple RNN dan LSTM berdasarkan nilai BLEU-4, METEOR, waktu training, waktu evaluasi, grafik loss, serta hasil qualitative analysis. Kedua model menggunakan dataset Flickr8k, preprocessing caption, vocabulary, feature CNN, dan konfigurasi eksperimen yang sama sehingga perbandingan difokuskan pada perbedaan kemampuan decoder dalam memproses sequence caption.

Berdasarkan hasil pengujian, LSTM secara umum menghasilkan performa yang lebih baik dibandingkan Simple RNN. Pada konfigurasi terbaik, RNN memperoleh BLEU-4 sebesar 0.127567 dan METEOR sebesar 0.324534, sedangkan LSTM memperoleh BLEU-4 sebesar 0.157926 dan METEOR sebesar 0.357980. Hal ini menunjukkan bahwa LSTM lebih mampu menangkap konteks kata dalam caption dibandingkan Simple RNN.

	num_layers	hidden_size	BLEU_4_rnn	BLEU_4_lstm	METEOR_rnn	METEOR_lstm
0	1	512	0.127567	0.157926	0.324534	0.357980
1	3	512	0.123389	0.126351	0.312528	0.325143
2	2	512	0.116353	0.142241	0.310350	0.330821
3	2	128	0.108400	0.113183	0.302030	0.301490
4	3	128	0.108164	0.112747	0.293262	0.299218
5	1	128	0.103690	0.105577	0.292966	0.299974

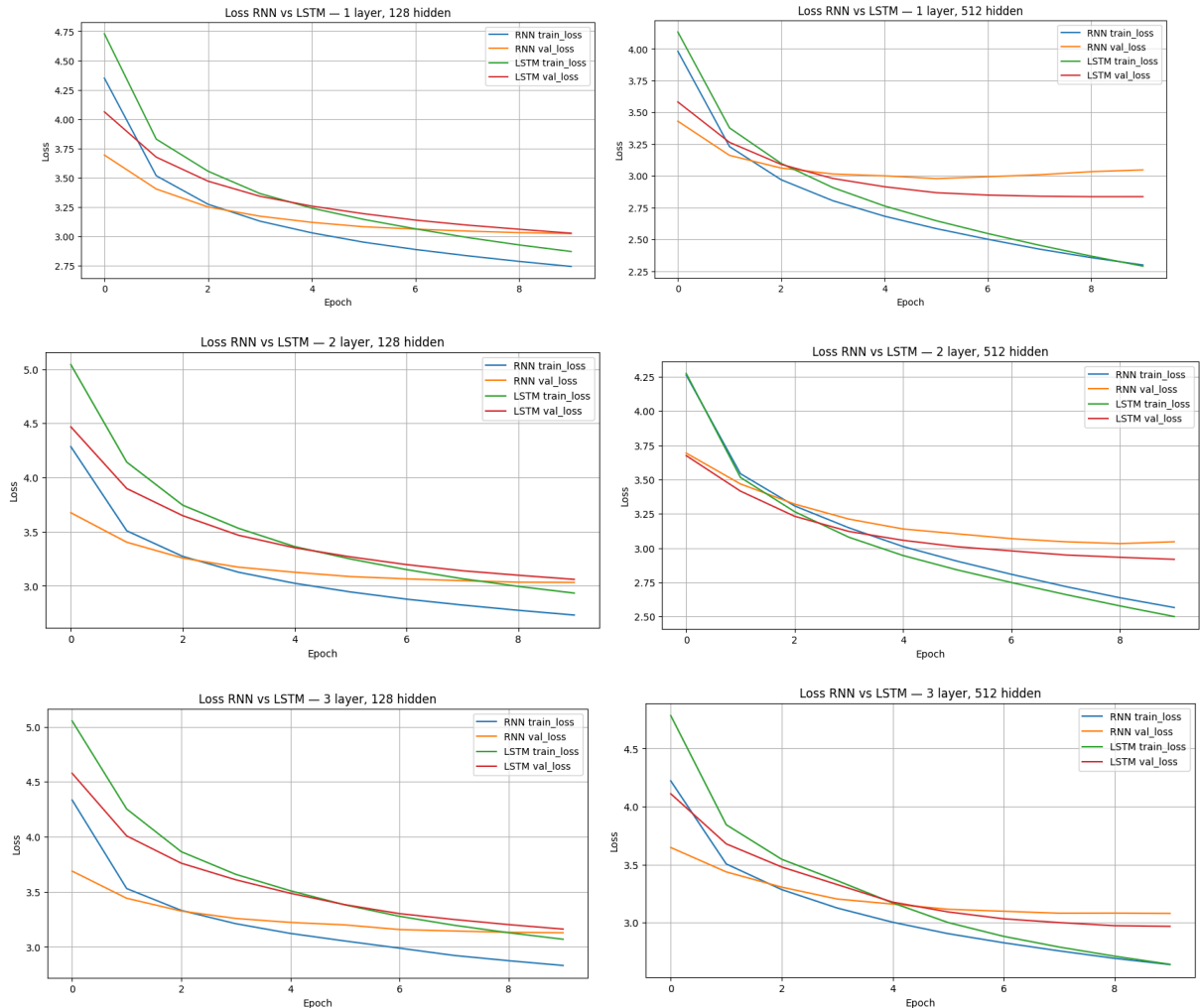
best_val_loss_rnn	best_val_loss_lstm	train_time_sec_rnn	train_time_sec_lstm	eval_time_sec_rnn	eval_time_sec_lstm
2.978200	2.836015	663.736187	1031.337243	1612.186594	761.344919
3.080303	2.968674	1212.576040	2261.770083	716.024523	888.055025
3.032977	2.918147	971.932288	1670.788568	1579.042459	825.040261
3.031727	3.059749	238.344616	340.966963	1842.950079	698.441146
3.127512	3.160627	328.186380	413.047829	778.080486	726.970896
3.024400	3.027315	226.468763	289.399894	1679.404727	676.961346

Dari sisi waktu eksekusi, LSTM cenderung membutuhkan waktu training yang lebih lama dibandingkan RNN karena arsitekturnya lebih kompleks. LSTM memiliki cell state dan beberapa gate sehingga komputasinya lebih besar daripada Simple RNN yang hanya menggunakan hidden state. Namun, peningkatan waktu tersebut sebanding dengan peningkatan kualitas caption yang dihasilkan, terutama pada konfigurasi hidden state 512.



Berdasarkan grafik training loss dan validation loss, kedua model menunjukkan pola loss yang menurun selama training. Namun, LSTM cenderung memiliki performa validasi

yang lebih stabil pada beberapa konfigurasi. Hal ini menunjukkan bahwa mekanisme gate pada LSTM membantu model mempertahankan informasi penting dalam sequence lebih panjang, sedangkan Simple RNN lebih mudah kehilangan konteks karena keterbatasan memori jangka panjang.



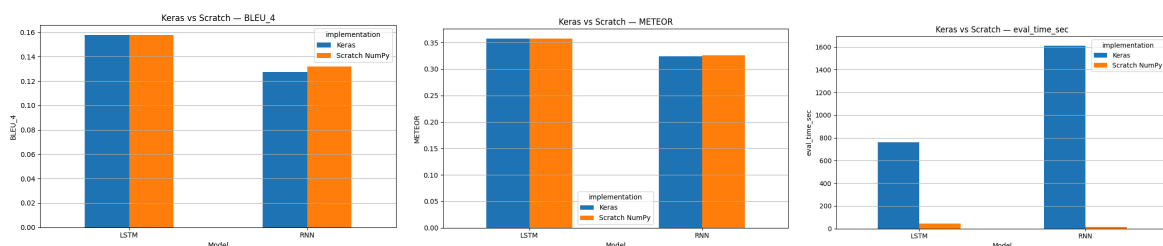
Pada qualitative analysis, hasil caption dari LSTM umumnya lebih lengkap dan lebih relevan terhadap objek utama pada gambar. RNN cenderung menghasilkan caption yang lebih umum dan sederhana, misalnya hanya menangkap objek seperti “man”, “dog”, atau “people”, tetapi kurang tepat dalam menjelaskan aktivitas atau detail gambar. Sebaliknya, LSTM lebih sering menghasilkan caption yang memiliki struktur kalimat lebih baik dan lebih sesuai dengan ground truth. Namun, pada beberapa gambar sederhana, RNN dan LSTM dapat menghasilkan caption yang mirip karena objek visualnya mudah dikenali.

	image	rnn_caption	lstm_caption	ground_truth	rnn_bleu4	lstm_bleu4	avg_bleu4
0	2135502491_a15c6b5eae.jpg	a brown and white dog runs through a field	a dog is running through the snow	A beautiful brown and white St Bernard running...	0.343295	0.840896	0.592095
1	2056930414_d2b0f1395a.jpg	a young boy in a blue shirt is playing with a toy	a little boy is playing with a toy car	A child plays a red toy guitar and sings into ...	0.356551	0.772551	0.564551
2	2260560631_09093be4c6.jpg	a dog is jumping over a low cut <unk>	a dog is running through a grassy area	A dirty dogs runs through the woods . A dog ...	0.204351	0.765206	0.484778
3	1267711451_e2a754b4f8.jpg	a black and white dog jumping over a hurdle	a black and white dog is running through a gra...	A black and white dog carrying a large stick ...	0.467138	0.453052	0.460095
4	1446933195_8fe9725d62.jpg	a dog is running through a grassy field	a dog is running through a grassy area	A black puppy is biting a tree limb . A brow...	0.089563	0.111477	0.100520
5	1897025969_0c41688fa6.jpg	a black dog is running through a grassy field	two black dogs are playing with a toy	A black dog and a tan dog fighting . A black...	0.155273	0.044761	0.100017
6	124972799_de706b6d0b.jpg	a dog is running through a grassy field	a dog is playing with a ball	A cat under the bench hissing with a dog growl...	0.078826	0.115879	0.097353
7	2295750198_6d152d7ceb.jpg	a young girl in a pink bathing suit is jumping...	a boy in a blue shirt is running through the w...	A girl is standing on one leg in the middle of...	0.027523	0.032297	0.029910
8	2286823363_7d554ea740.jpg	a boy in a blue shirt is jumping on a trampoline	a boy in a blue shirt is standing in front of ...	A little boy jumping from one chair to another...	0.032297	0.024870	0.028583
9	225699652_53f6fb33cd.jpg	a man is standing on a boat overlooking a lake	a man is climbing a large rock while another m...	Two dolphins flying headfirst into a beautiful...	0.027896	0.021598	0.024747

Secara keseluruhan, LSTM lebih unggul dibandingkan Simple RNN untuk task image captioning karena memiliki kemampuan menyimpan informasi jangka panjang melalui cell state dan gate. Selain itu, arsitektur LSTM juga lebih mampu mengatasi permasalahan *vanishing gradient* yang sering terjadi pada Simple RNN ketika menghadapi sekuens yang panjang. Pada Simple RNN, informasi dari timestep sebelumnya cenderung semakin melemah seiring bertambahnya panjang sekuens sehingga model lebih mudah kehilangan konteks penting saat menghasilkan caption. Akibatnya, kualitas caption yang dihasilkan menjadi kurang konsisten terutama pada kalimat yang lebih panjang. Meskipun kemampuan memorinya lebih terbatas, Simple RNN juga memiliki kelebihan yaitu arsitekturnya yang lebih sederhana dan relatif lebih ringan.

3.2.6. Perbandingan Keras vs From Scratch

Pada eksperimen ini, dilakukan perbandingan hasil prediksi antara model Keras dengan model *from scratch* yang telah kami implementasikan. Kedua model menggunakan bobot hasil training, vocabulary, dan parameter evaluasi yang sama sehingga perbandingan dilakukan pada proses forward propagation saja.



Pada RNN, diperoleh nilai BLEU-4 dan METEOR yang similar untuk kedua model yaitu 0.127567 dengan 0.131845 dan 0.324534 dengan 0.326439. Hal ini membuktikan bahwa implementasi from scratch untuk forward propagation pada RNN berhasil dan sudah

berjalan dengan baik. Perbedaan utama hanya terlihat pada waktu evaluasi, di mana Keras memiliki waktu eksekusi yang lebih lambat dibandingkan implementasi from scratch.

	model_type	implementation	BLEU_4	METEOR	eval_time_sec	num_images
0	RNN	Keras	0.127567	0.324534	1612.186594	1000
1	RNN	Scratch NumPy	0.131845	0.326439	15.079772	1000

Pada LSTM, diperoleh nilai BLEU-4 dan METEOR yang juga hampir sama untuk kedua model yaitu 0.1579 dan 0.357. Hal ini membuktikan bahwa implementasi from scratch untuk forward propagation pada LSTM berhasil dan sudah berjalan dengan baik. Perbedaan utama hanya terlihat pada waktu evaluasi, di mana Keras memiliki waktu eksekusi yang lebih lambat dibandingkan implementasi from scratch.

	model_type	implementation	BLEU_4	METEOR	eval_time_sec	num_images
0	LSTM	Keras	0.157926	0.357980	761.344919	1000
1	LSTM	Scratch NumPy	0.157962	0.357861	45.459813	1000

3.2.7. Pengaruh panjang maksimum caption terhadap BLEU-4

Pengujian panjang maksimum caption pada RNN menunjukkan bahwa semakin besar max caption length, nilai BLEU-4 menjadi semakin rendah. Hal ini dapat dilihat pada gambar, model dengan max caption length 10 menghasilkan BLEU-4 tertinggi sebesar 0.138896, sedangkan pada panjang 20 dan 30 nilainya menurun tipis menjadi 0.131845 dan 0.131815.

	model_type	max_caption_len	BLEU_4	METEOR	eval_time_sec	num_images
0	RNN	10	0.138896	0.316820	12.112760	1000
1	RNN	20	0.131845	0.326439	17.424380	1000
2	RNN	30	0.131815	0.326434	16.479763	1000

Pengujian panjang maksimum caption pada LSTM menunjukkan pola yang sama bahwa semakin besar max caption length, nilai BLEU-4 menjadi semakin rendah. Hal ini dapat dilihat pada gambar, model dengan max caption length 10 menghasilkan BLEU-4 tertinggi sebesar 0.166839, sedangkan pada panjang 20 dan 30 nilainya menurun menjadi 0.157962 dan 0.158095.

	model_type	max_caption_len	BLEU_4	METEOR	eval_time_sec	num_images
0	LSTM	10	0.166839	0.343894	38.124515	1000
1	LSTM	20	0.157962	0.357861	49.279828	1000
2	LSTM	30	0.158095	0.358021	55.843523	1000

Hal ini menunjukkan bahwa penambahan panjang maksimum caption tidak selalu meningkatkan kualitas caption yang dihasilkan. Caption yang terlalu panjang justru dapat menambah kompleksitas sekuens dan memperbesar kemungkinan model menghasilkan prediksi kata yang kurang relevan.

BAB 4

KESIMPULAN DAN SARAN

4.1. Kesimpulan

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, dapat disimpulkan bahwa model CNN, RNN, dan LSTM yang dibangun telah berhasil diimplementasikan dengan baik dan mampu melakukan proses klasifikasi dan *image captioning* pada dataset yang digunakan. Selain itu, implementasi from scratch yang dibuat juga berhasil menghasilkan output dan performa yang konsisten dengan implementasi berbasis framework sehingga membuktikan bahwa proses forward propagation yang diimplementasi telah berjalan dengan benar.

Pada pengujian CNN dapat disimpulkan bahwa performa ketika menggunakan *shared parameter* jauh lebih cepat dan hasil F1 score nya lebih baik jika dibandingkan menggunakan *non-shared* parameter. Model dengan *non-shared* parameter lebih berisiko *overfitting* karena model menghafal setiap detail fitur sehingga ketika diberikan data tes yang berbeda performanya menurun. Performa CNN juga dipengaruhi oleh jumlah layer, jumlah filter, ukuran filter, dan jenis *pooling*. Berdasarkan pengujian yang dilakukan, dapat disimpulkan bahwa jumlah layer, jumlah filter, dan ukuran filter memiliki hubungan berbanding lurus. Semakin tinggi nilainya maka performa F1 score yang dihasilkan juga semakin baik. Dilihat dari jenis *pooling*, *max pooling* cenderung memiliki performa yang lebih baik dibandingkan dengan *average pooling*. Akan tetapi, pola pengamatan yang ditemukan tidak sepenuhnya ter-*cluster* dengan sempurna. Terdapat beberapa konfigurasi yang performanya berbeda dari *cluster* yang seharusnya. Hal ini juga dipengaruhi oleh dataset yang digunakan dan faktor kombinasi yang dapat menciptakan performa lebih baik ketika digabungkan.

Hal serupa juga terlihat pada pengujian parameter model RNN & LSTM. Performa model dipengaruhi oleh beberapa faktor sekaligus, seperti jumlah layer dan ukuran hidden state. Pada beberapa eksperimen, penambahan layer atau ukuran hidden state mampu meningkatkan performa BLEU-4 dan METEOR, namun pada konfigurasi lain justru menyebabkan performa menurun. Hal ini menunjukkan bahwa performa model sangat bergantung pada karakteristik dataset serta kombinasi hyperparameter yang digunakan. Sehingga tidak ada pola global yang berlaku pada seluruh konfigurasi.

Dalam konteks task image captioning ini, model LSTM secara umum menghasilkan performa yang lebih baik dibandingkan Simple RNN. Hal ini karena LSTM memiliki mekanisme cell state dan gate yang memungkinkan model mempertahankan informasi penting dalam jangka panjang serta mengurangi permasalahan vanishing gradient yang sering terjadi pada Simple RNN. Sehingga LSTM mampu mempelajari hubungan antar kata pada caption dengan lebih baik dan menghasilkan nilai BLEU-4 serta METEOR yang lebih tinggi. Meskipun demikian, LSTM memberikan trade off berupa kompleksitas komputasi yang jauh lebih besar dibandingkan RNN karena jumlah parameter dan operasi matematis yang digunakan lebih banyak.

4.2.Saran

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, terdapat beberapa saran yang dapat dipertimbangkan untuk pengembangan lebih lanjut. Pertama, eksperimen dapat dikembangkan dengan menggunakan variasi yang lebih luas seperti pada konfigurasi arsitektur model yang digunakan.

Keterbatasan *resource* laptop membuat proses *training* memakan waktu yang sangat lama. Hal ini menyebabkan kami kesulitan ketika ingin mencoba *tuning model* yang lain. Proses *training* juga dilakukan dengan *epoch* yang tidak terlalu banyak sehingga nilai evaluasinya belum konvergen. Saran terkait dataset yang digunakan, akan lebih baik jika jumlahnya tidak terlalu besar supaya kami bisa lebih fokus dalam meningkatkan performa model dibandingkan menunggu proses *training* yang begitu lama.

PEMBAGIAN TUGAS

NIM	Nama	Pembagian Tugas
13523078	Anella Utari Gunadi	Implementasi RNN, membuat laporan, pengujian RNN
13523079	Nayla Zahira	Implementasi RNN, membuat laporan, pengujian LSTM
13523080	Diyah Susan Nugrahani	Implementasi CNN, membuat laporan, pengujian CNN

REFERENSI

Convolutional Neural Network (CNN)

https://d2l.ai/chapter_convolutional-neural-networks/index.html

Long Short-Term Memory (LSTM)

https://d2l.ai/chapter_recurrent-modern/lstm.html

Dataset Intel Image Classification

<https://www.kaggle.com/datasets/puneet6060/intel-image-classification>

Dataset Flickr8k

<https://www.kaggle.com/datasets/adityajn105/flickr8k>

LAMPIRAN

Bobot pembelajaran : Tubes 2 ML_Kelompok 9

Lampiran Form Penggunaan AI

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	-
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	-
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Menggunakan Gemini dan Copilot untuk <i>debugging</i> kode yang error dan memperdalam konsep sebelum implementasi
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	-
Lainnya, Jelaskan:		-